

DA660 - AI and Vector Search

Basics

Student: Firstname Lastname

Version: 2.0



Contents

1. Introduction	4
2. Setup	4
2.1 Initialize & Configure Python	4
2.1.1 Verify Python v3.12 is installed	4
2.1.2 Activate Python virtual environment	4
2.1.3 Ensure Required Libraries are Installed	5
2.3 Getting the AI access token	5
2.4 Atlas Cluster	6
3. Concepts	6
3.1 Introduction to Vectors and Vector Search	6
3.1.1 Understanding Vectors	6
3.1.1.1 What are Vectors?	6
3.1.1.2 Vector Applications in AI	7
3.1.1.2.1 Word Embeddings	7
3.1.1.2.2 Semantic Search	8
1. Euclidean Similarity Search:	8
2. Cosine Similarity Search:	8
3. Dot Product Similarity Search:	8
3.1.1.2.3 Generative AI	9
3.1.2 Types of Vectors	9
3.1.2.1 Sparse Vectors	9
3.1.2.2 Dense Vectors	10
3.1.2.3 Sparse vs. Dense Vectors: Key Differences	11
3.1.2.4 Applications in Vector Search and Generative AI	11
3.2 Approximate Nearest Neighbours (ANN) Search	12
3.2.1 How does it work?	13
3.2.2 Applications	14
3.2.2.1 Image and video recognition	14
3.2.2.2 Recommendation systems	14
3.2.2.3 Natural language processing	14
3.2.2.4 Anomaly detection	14
3.2.2.5 Reinforcement learning	14
3.2.2.6 Retrieval Augmented Generation (RAG)	14
3.2.3 Exercise	14
3.2.4 Conclusion	17
3.3 Hierarchical Navigable Small Worlds (HNSW) Algorithm	18

3.3.1 What is the HNSW Algorithm?	18
3.3.2 Why is HNSW important in vector search?	19
3.4 Text Embeddings and Chunking Techniques	19
3.4.1 Text Embeddings for Semantic Search	19
3.4.1.1 Introduction to Text Embeddings	20
3.4.1.1.1 Word2Vec	20
3.4.1.1.2 GloVe	20
3.4.1.1.3 FastText	20
3.4.1.1.4 BERT	20
3.4.1.2 Semantic Search and Text Embeddings	21
3.4.1.3 Vector Embedding Indexing	21
3.4.1.4 Practical Applications and Challenges	21
Retrieve embeddings for text	22
3.4.2 Chunking Techniques	22
3.4.2.1 What is Chunking?	22
3.4.2.2 Why is Chunking Important?	23
3.4.2.3 Exercise	24
3.4.3 Chunking Techniques	26
3.4.3.1 Exercise: Chuck and Embed	27
3.5 Using Atlas Vector Search	32
3.5.1 Building Movie Recommendation app	32
3.5.2 Create Atlas Vector Search indexes	32
3.5.3 Perform search	35
3.5.3.1 Preparing a Python application for searching	35
3.5.3.2 The Vector Search Query	38
3.5.3.2 Exercise	39
3.5.4 Perform Image search with different embedding	41
3.5.4.1 Image search using a different embedding	41
3.5.4.2 Challenge: Similar Image Search	47
3.5.5 MongoDB Atlas Developer Data Platform	51
3.5.5.1 Using Atlas Database Triggers to create vector embeddings	51
3.6 MongoDB's applications as a Vector Database	54
3.6.1 Large Language Models (LLMs)	54
3.6.1.1 How do LLMs work?	54
3.6.1.2 How can LLMs leverage MongoDB?	54
3.6.1.3 Applications of LLMs	56
3.6.2 Retrieval-augmented generation (RAG) Architecture	57
3.6.2.1 Use Case: Customized Chatbot using MongoDB & RAG	59



3.6.2.2 How does RAG help LLMs?	60
3.6.2.3 Exercise: RAG Code Review and Discussion	60
Setup	62
Load Data into Atlas	62
Create Vector Search Index	63
Pass Vector Search results to GPT	63
4. Vector Search Nodes	66
5. Conclusion	67
Solutions and Troubleshooting	67
Embedded VS Code	67
Similar Image search	68

How to use this document

Answer Block; the answer block may contain partial information that requires modifications by the student

Important / Note

Warning or critical call out

Code block for student execution without modification

Confirmation Code / Needed Information for the section



1. Introduction

This is a fundamental level course introducing Vector Search and the relevant basic concepts, with focus on semantic search.

MongoDB in the AI world offers a powerful way to organize and retrieve massive amounts of information based on the relationships between data points. This data can be used to generate contextually relevant responses by retrieving similar examples from a vast repository of text, like OpenAI's . Another fascinating application is where the core features of Atlas Vector Search can be used to enhance the process of generating natural language responses in chatbots and virtual assistants.

By understanding the fundamentals of Atlas Vector Search, you can contribute to cutting-edge technologies that are revolutionizing the way we interact with AI systems. Learners will acquire knowledge about vectors, ANN, HNSW algorithms, text embeddings, chunking techniques, and stitching it all together with Atlas Vector Search by building a practical use case.

2. Setup

2.1 Initialize & Configure Python

- The examples and exercises in this course use Python.
- For the Atlas Vector Search exercises, we will use code examples in Python.
- Dependencies are pre-installed.

2.1.1 Verify Python v3.12 is installed

```
# python3 --version  
Python 3.12.3
```

2.1.2 Activate Python virtual environment

The code requires around 5.2Gb of external libraries. These are already installed in the folder `/root/da660/venv/`. Configure this as a virtual Python environment, and then activate it. You can ensure your Python virtual environment is activated by looking for “(venv)” on the left of the terminal.

```
# Select the virtual environment
python3 -m venv /root/da660/venv/
# Activate the virtual environment
source /root/da660/venv/bin/activate
```

2.1.3 Ensure Required Libraries are Installed

```
python3 -m pip install -r
/root/da660/mongodb-atlas-vector-search-image-products/requirements.txt

python3 -m pip install -r
/root/da660/atlas-vector-search-rag/requirements.txt
```

Please note that if you encounter a "ModuleNotFoundError" error, ensure that the Python virtual environment is activated.

2.3 Getting the AI access tokens

MongoDB's Azure OpenAI deployment is used in this training to address the requests to the an LLM.

- The MDB Azure deployment will require that the request includes an API key token.
- The Training team will provide you with this token at the start of this course.

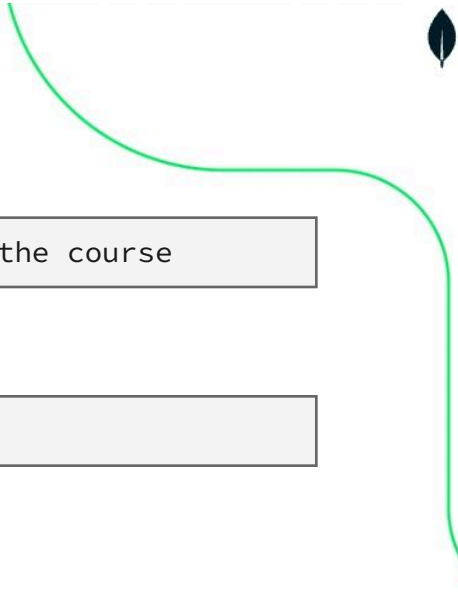
Please note that the token expires at the end of the course

Run this code to store the token in an environment variable.

```
export AZURE_OPENAI_API_KEY=<paste the token here>
```

VoyageAI is used in this training to host an embedding model that will convert text into vectors.

- VoyageAI requires that the request includes an API key token.
- The Training team will provide you with this token at the start of this course.



```
Please note that the token expires at the end of the course
```

Run this code to store the token in an environment variable.

```
export VOYAGE_AI_KEY=<paste the token here>
```

2.4 Atlas Cluster

The course makes use of Atlas Cluster, loaded with the sample dataset. If you do not already have this available, follow the steps below.

- [Creating an Atlas cluster](#) (For this training, we will use an M10 cluster*)
- [Loading sample data set](#)

3. Concepts

3.1 Introduction to Vectors and Vector Search

3.1.1 Understanding Vectors

In this subsection, we'll learn about vectors, their properties, and how they play a crucial role in powering semantic search and its applications.

3.1.1.1 What are Vectors?

In the context of AI, vectors are mathematical objects that are commonly used to represent and manipulate data. They are often used to represent features or attributes of an object or a data point.

A vector can be thought of as a series of numbers arranged in a specific order. Each number in the vector represents a certain dimension or property of the object being represented. For example, if we are representing a person's height and weight as a vector, we can have a 2-dimensional vector with the first number representing height and the second number representing weight.

Representing data (like text, images, videos, users, music, etc) as points in n-dimensional space, allows for similar data to be represented physically close together within that space.

Let's practise with some basic examples, just to understand these concepts. Imagine we have some words that we would like to represent as vectors: cat, dog, table, chairs, woman, men, girl, snakes.

We could classify these words using a semantic approach based on different categorizations based on the word's meaning. For instance, let's add some categories as columns and classify the words in each row with 1s or 0s if the word could fall in that category bucket or not respectively:

	Object	Creature	Female	Male	Young	Human	Feline	Reptile
cat	0	1	1	1	0	0	1	0
dog	0	1	1	1	0	0	0	0
table	1	0	0	0	0	0	0	0
chairs	1	0	0	0	0	0	0	0
woman	0	1	1	0	0	1	0	0
men	0	1	0	1	0	1	0	0
girl	0	1	1	0	1	1	0	0
snakes	0	1	1	1	0	0	0	1

If we encode these words as vectors based on this 8 category representation, the word "cat" is embedded as $[0,1,1,1,0,0,1,0]$. If we use the same embedding model to encode the word "tiger", we would get a similar vector embedding: $[0,1,1,1,0,0,1,0]$. Comparing the vectors allows us to see the similarities between words.

Using this model, encode the word "boys" into a vector. What similarities do you get?

This was just a basic example to understand this encoding concept. The embedding model we defined had only 8 classifications and the value in each is a boolean value (if it could be classified in the respective category or not respectively). Instead of discrete values, we could use a continuous scale from -1 to 1 representing the probability that we can classify the word in the category. This allows us to capture finer details:

	Object	Creature	Female	Male	Young	Human	Feline	Reptile
cat	-0.78	0.79	0.15	0.20	0.75	-0.68	0.98	-0.5

3.1.1.2 Vector Applications in AI

Vectors play a significant role in various AI applications, including semantic search and large language models. Here's how:



3.1.1.2.1 Word Embeddings

In natural language processing (NLP), words are often represented as high-dimensional vectors called word embeddings. These vectors capture the semantic meaning of words, allowing algorithms to understand and process text data more effectively.

3.1.1.2.2 Semantic Search

By representing text documents and search queries as vectors, we can measure the similarity between them using vector operations like the dot product or cosine similarity. This enables us to perform a semantic search, where the search engine returns results based on the meaning of the query rather than just keyword matching.

Atlas Vector Search supports the following types of similarity searches:

1. Euclidean Similarity Search:

Imagine you have a dataset of houses with attributes such as size (in square feet or metres squared) and price (in dollars). Now, you want to find houses that are similar in terms of their size and price. Euclidean similarity search calculates the distance between two data points in a multi-dimensional space. In our example, it measures the straight-line distance between two houses in terms of their size and price to determine how similar they are. The closer the distance, the more similar the houses are. Obviously there is a lot more involved in comparing houses, but this is just a simple example: in reality we would include a lot more attributes.

The mathematical formula for the Euclidean distance between two n-dimensional vectors is:

$$\sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \dots + (a_n - b_n)^2}$$

2. Cosine Similarity Search:

Consider a document search system. Users enter short lines of text (queries) and the system must find a document which best matches the query. Cosine similarity is a measure of the difference in angle between two vectors, regardless of their magnitudes. The smaller the difference, the more similar the vectors are. A document and a search query whose vectors have a cosine similarity of close to 1 indicate high similarity.¹

The mathematical formula for the cosine of two vectors is:

$$\frac{a \cdot b}{|a| \cdot |b|}$$

¹ <https://www.datastax.com/guides/real-world-applications-of-cosine-similarity>



3. Dot Product Similarity Search:

Now, let's say you are building a music recommendation system. You have a dataset of songs, and each song is represented by features like genre and tempo. Dot product similarity search takes into account the cosine similarity (angle between) but also the magnitude of the vectors. Some data models will generate longer vectors for examples which appear regularly in the training set, so longer vectors could be an indication of popularity. With Dot Product similarity we favour a vector with a greater magnitude over a vector with an identical angle².

The mathematical formula for the dot product of two n-dimensional vectors is:

$$a_1 b_1 + a_2 b_2 + \dots + a_n b_n = |a||b|\cos(\theta)$$

3.1.1.2.3 Generative AI

The use of vectors in generative AI models like GPT-4 plays a crucial role in capturing context, maintaining coherence, and generating meaningful responses in natural language conversations. These vectors capture meaning of the input text, allowing the model to generate coherent and contextually relevant responses.

As you progress through this course, you'll gain a deeper understanding of how vectors are used in various AI applications, and you'll be able to harness their power to build more advanced models and systems.

3.1.2 Types of Vectors

In this sub-section, we'll dive deep into the concepts of sparse and dense vectors, their differences, and their applications in vector search and generative AI.

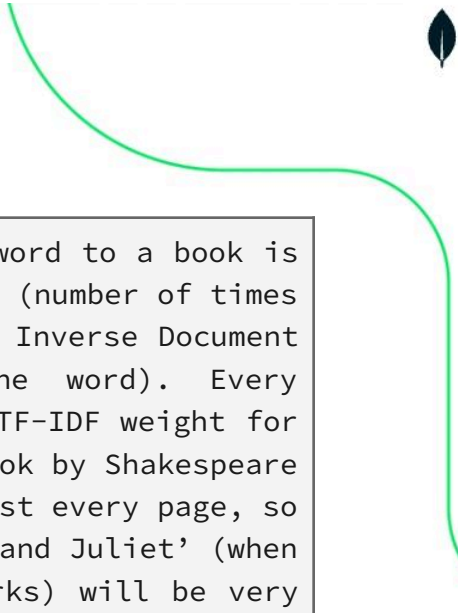
3.1.2.1 Sparse Vectors

Sparse vectors are high-dimensional vectors with a majority of their elements being zero. In other words, they have a lot of "empty space" or sparsity. Let's take a look at an example:

```
Sparse Vector: [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

As you can see, only one element in this 20-dimensional vector is non-zero. Sparse vectors are commonly used in natural language processing (NLP) tasks, such as text classification and information retrieval. They are particularly useful for representing high-dimensional data, like word frequency counts in a large vocabulary, meaning they are suitable for keyword searches.

² <https://developers.google.com/machine-learning/clustering/similarity/measuring-similarity>



As a matter of interest, the ‘importance’ of a word to a book is calculated as a [TF-IDF](#) weight: the Term Frequency (number of times the word is seen in the book) multiplied by the Inverse Document Frequency (number of books which contain the word). Every [English] book contains the word ‘the’, so the TF-IDF weight for ‘the’ will be close to zero. However, only one book by Shakespeare contains the word ‘Romeo’, and it’s there on almost every page, so the TF-IDF weight for ‘Romeo’ in the book ‘Romeo and Juliet’ (when considered against a corpus of Shakespeare’s works) will be very high.

Example

Imagine you are looking for a book about polar bears. The words ‘polar’ and ‘bear’ should occur in that book relatively often. And most other books will contain neither the word ‘polar’ nor ‘bear’. Therefore, doing a straight keyword search will most likely return relevant results, and should be relatively performant, as most books can be simply excluded from the search.

In this analogy, each book is a vector, each ‘dimension’ represents a dictionary word, and the value of each dimension represents how important the word is in the book. The vector representing our polar bear book will have high values for the dimensions representing the words ‘polar’ and ‘bear’, and most likely zeroes for the dimensions representing the words ‘jungle’, ‘parrot’, and ‘equator’. The high number of zeros is what makes the vector “Sparse”.

3.1.2.2 Dense Vectors

Dense vectors, on the other hand, are vectors with most of their elements being non-zero. They are more “packed” with information compared to sparse vectors. Here’s an example:

Dense Vector: [0.12, -0.45, 0.67, 0.89, -0.34, 0.56, -0.71, 0.29, 0.49, -0.21]



In this 10-dimensional vector, all elements have non-zero values. Dense vectors are often used in machine learning and deep learning models, such as neural networks, to represent features or embeddings.

Example

Imagine that you are looking for a book about ... well, you're not exactly sure what they are called, but you're pretty sure they're 'big animals' that 'live in the cold' and 'hunt seals' (try entering those search terms into Google and see what you get!). A keyword search won't help you here, so you will have to search for meaning instead. This is much more complex, so you'll need a much greater amount of nuanced information.

A dense vector would need to be generated using a neural net, which has been trained on large amounts of data. In a dense vector, each dimension contains relevant information about the semantics of the text, which allows us to identify patterns based on semantics, not just syntax.

3.1.2.3 Sparse vs. Dense Vectors: Key Differences

Sparse	Dense
Sparse vectors have a high proportion of elements with zero values	Dense vectors have mostly non-zero elements
Sparse vectors can be represented more efficiently in memory using data structures like dictionaries or compressed sparse row (CSR) format, which only store non-zero elements	Dense vectors require more memory as they store all elements
Operations on sparse vectors can be faster due to the presence of fewer non-zero elements	Some algorithms and data structures are optimised for dense vectors, making them more efficient in certain cases

3.1.2.4 Applications in Vector Search and Generative AI

Sparse and dense vectors play a crucial role in vector search and generative AI. Here's how:

1. In semantic search, documents and queries are often represented as high-dimensional vectors. Sparse vectors can be used to represent term frequency-inverse document frequency (TF-IDF) weights, while dense vectors can be used for word embeddings or document embeddings generated by neural networks.
2. In generative models like GPT-4, dense vectors are used to represent the hidden states and embeddings of tokens. These dense vectors are like a bag of words, but they have more information in them. They capture the meaning of words and phrases, which allows the model to generate text that is coherent and contextually relevant.

Hidden States

Within a neural network, especially in recurrent or transformer models like GPT, "hidden states" are the internal representations of the model's understanding of the input data. These states contain information about the context and semantic meaning of the input.

Imagine that you are trying to write a story about a dog. You could start by writing down a list of words that are related to dogs, such as "fur", "tail", and "bark". However, this would not be enough to write a good story. You would also need to understand the meaning of these words and how they relate to each other.

Dense vectors are like the meaning of words. They capture the relationships between words and phrases, which allows the model to generate text that is coherent and contextually relevant.

So, in simple terms, dense vectors are used to represent the internal states (hidden states) and the meaning of individual words or tokens (embeddings) in the text that the model processes. These dense vectors are essential for the model to understand and generate human-like text.

By now, you should have some basic understanding of sparse and dense vectors, their differences, and their applications in vector search and generative AI. Next we will talk about the algorithms involved in Atlas Vector Search concepts to enhance your knowledge and skills in this exciting domain!

3.2 Approximate Nearest Neighbours (ANN) Search

Approximate nearest neighbour (ANN) search is a technique used in the field of artificial intelligence and data mining to find approximate nearest neighbours to a given query point in a large dataset. Atlas Vector Search's \$vectorSearch stage performs ANN searches on vector fields.

The goal of ANN is to efficiently find points in a dataset that are close in distance to a given query point, in this case, vector. The term "approximate" indicates that the algorithm may not find the exact nearest neighbours, but instead returns a set of points that are close enough to be considered as potential nearest neighbours.



ANN algorithms are particularly useful in scenarios where the dataset is too large to perform an exhaustive search for the exact nearest neighbours. These algorithms use different techniques, such as data structures, indexing methods, and heuristics, to quickly identify potential nearest neighbours without examining every point in the dataset.

By using ANN techniques, it is possible to speed up the search process and make it feasible to perform similarity-based queries on large datasets in real-time or near-real-time. ANN has applications in various domains, such as information retrieval, recommendation systems, computer vision, and pattern recognition.

It's worth mentioning that the accuracy of the approximate nearest neighbour search depends on the specific algorithm and parameters chosen. The trade-off usually lies between efficiency and accuracy, meaning that higher accuracy comes at the cost of longer search times.

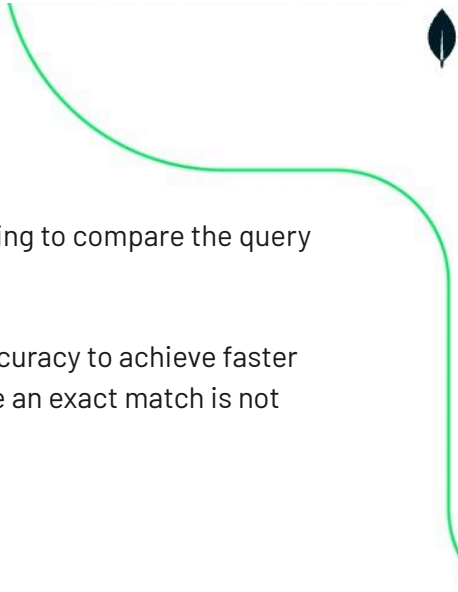
Note that Atlas Vector Search also supports Exact Nearest Neighbor if more accuracy is required. This could be useful when benchmarking or when using small-scale vector data.

3.2.1 How does it work?

To understand how ANN search works, let's consider the example of a large dataset of images, and you want to find the similar images to a particular query image. An ANN search goes through the following steps:

1. **Preprocessing:** In the preprocessing step, the dataset is transformed into a more suitable format for efficient searching. This typically involves converting the images into a feature representation that captures their characteristics, i.e. into vectors.
2. **Searching:** When a query image is given, the search algorithm starts from the root of the search index and traverses, comparing the query image's features with the features of the data points at each node. Based on the similarities, it decides which step to go to next. This process continues until it reaches the last item in the data set or a certain stopping criterion is met.
3. **Approximate Nearest Neighbours:** While searching, a set of potential nearest neighbour candidates is obtained. These candidates may not be the exact nearest neighbours, but they are close enough to be considered.
4. **Finally,** a post-processing step is performed on these candidates to further refine the search and find the closest matches to the query image.

The key idea behind ANN search is to strategically explore the data set to prioritize the search in regions that are more likely to contain the nearest neighbours, while skipping irrelevant regions.



This allows for efficient searching in high-dimensional spaces without having to compare the query image with all the data points in the dataset.

However, it's important to understand that ANN search sacrifices some accuracy to achieve faster search times. The approximation is acceptable in many applications where an exact match is not required, and the speed gain is valuable.

3.2.2 Applications

3.2.2.1 Image and video recognition

ANN algorithms can be used to quickly find similar images or videos in large databases. This is useful in applications such as content-based image retrieval, where you want to find images that are visually similar to a query image or text. See a demonstration [here](#).

3.2.2.2 Recommendation systems

ANN algorithms can be used to find similar items or users in recommendation systems. For example, if you are using a music streaming app and you like a particular song, the app can use ANN to recommend other songs that are similar in style or genre.

3.2.2.3 Natural language processing

ANN algorithms can be used to find similar documents or words in text data. This is useful in applications such as document clustering, where you want to group similar documents together.

3.2.2.4 Anomaly detection

ANN algorithms can be used to detect anomalies or outliers in datasets. For example, in credit card fraud detection, ANN can help identify transactions that deviate from typical patterns and may indicate fraudulent activity.

3.2.2.5 Reinforcement learning

ANN algorithms can be used in reinforcement learning to approximate the value function or policy. This helps in making decisions and taking actions in dynamic environments, such as in autonomous driving or game-playing agents.

3.2.2.6 Retrieval Augmented Generation (RAG)

RAG is an artificial intelligence technique that improves the quality of generative AI by allowing large language models (LLMs) to tap additional data resources without persisting it. It depends on retrieving the data from a data store, and then generates a response.

ANN plays a crucial role in RAG by helping to retrieve relevant information from a large database, allowing the language model to generate accurate responses. For example, let's say you have a question about a particular topic, and you want an AI system to generate a detailed answer. Instead

of searching through the entire database, ANN enables the system to quickly find the most relevant articles or documents. By using approximate methods, the system can efficiently identify the nearest neighbours or most similar examples related to your query.

3.2.3 Exercise

Implementing a full Approximate Nearest Neighbour algorithm is far beyond the scope of this course, but we can implement a Nearest Neighbour algorithm quite easily.

To get started, imagine you have a dataset of various points, where each point has an x and y coordinate. You want to find the closest point in the dataset to a given query point.

Create a file called **ann.py**, with the following content:

```
import sys
import math

data = [
    { "x": 10, "y": 20 },
    { "x": 30, "y": 40 },
    { "x": 50, "y": 45 },
    { "x": 90, "y": 100 }
]

def calculate_distance(point1, point2):
    delta_x = point2['x'] - point1['x']
    delta_y = point2['y'] - point1['y']
    return math.sqrt(delta_x ** 2 + delta_y ** 2)

def find_nearest_neighbor(query_point):
    closest_point = data[0] # assume that the first point is the
    # closest initially.
    min_distance = calculate_distance(query_point, closest_point)

    for point in data:
        distance = calculate_distance(query_point, point)
        if distance < min_distance:
            closest_point = point
            min_distance = distance

    return closest_point

# Test it out:
```



```

if len(sys.argv) < 3:
    query_point = {"x": 70, "y": 70}
else:
    query_point = {"x": int(sys.argv[1]), "y": int(sys.argv[2])}

nearest_neighbour = find_nearest_neighbor(query_point)
print(f"The nearest neighbour to the query point
({query_point['x']}, {query_point['y']}) is
({nearest_neighbour['x']}, {nearest_neighbour['y']}).")

```

ann.py file does the following tasks:

- *Defining the dataset*

The code defines a dataset of four points, each with an x and y coordinate.

- *Calculating the distance between two points*

The code defines a function called calculateDistance(), which calculates the Euclidean distance between two points.

- *Implementing the ANN algorithm*

The code defines a function called findNearestNeighbour(), which implements the ANN algorithm. The ANN algorithm takes one input: the query point. The algorithm then calculates the distances between the query point and all of the points in the dataset. It then sorts the distances in ascending order and returns the nearest neighbours.

- *Testing it out*

The code tests the ANN algorithm by using { x: 70, y: 70 } as the query point, and prints its output on the console. If you supply command line arguments, the code will use these as the x and y values respectively instead.

```

python3 ann.py
python3 ann.py 20 15

```

Predict the nearest point with the following inputs, and write your answer against the Nearest Neighbour here:

1. queryPoint = { x: 20, y: 15 }
Nearest Neighbour = <>
2. queryPoint = { x: 35, y: 40 }

```
Nearest Neighbour = <>
```

```
3. queryPoint = { x: 83, y: 75 }  
Nearest Neighbour = <>
```

Now, reimplement the ANN algorithm to deal with 3 dimensions. Use the following dataset to test:

```
data = [  
  { "x": 10, "y": 20, "z": 20 },  
  { "x": 30, "y": 40, "z": 60 },  
  { "x": 50, "y": 45, "z": 90 },  
  { "x": 90, "y": 100, "z": 120 }  
]
```

Try and predict the nearest neighbour with these inputs:

```
4. queryPoint = { x: 80, y: 60, z: 20 }  
Nearest Neighbour = <>  
  
5. queryPoint = { x: 10, y: 40, z: 130 }  
Nearest Neighbour = <>  
  
6. queryPoint = { x: 80, y: 75, z: 35 }  
Nearest Neighbour = <>
```

Most likely your success rate guessing the nearest neighbour in 3 dimensions went down a bit. Also, you will note that for the extra dimension, the distance calculation needed an additional 3 operations.

One of the vector sets we will use later in this workshop has 1536 dimensions per vector, and the full dataset has almost 3500 vectors.

What problems would you expect using the 'Nearest Neighbour' algorithm above to find the nearest neighbour for a vector with this many dimensions? Place your ideas in the block below:

```
High cpu; long runtimes; inefficient, exhaustive searches;
```

Assuming you can't reduce the size of the vectors or the size of the dataset, how might you go about addressing these problems?

Indexing, pre-sorting the data, pre-building a nearest-neighbour graph, limiting the number of calculations (reducing accuracy)

3.2.4 Conclusion

The algorithm shown here has been simplified to perform an exhaustive search through the dataset. With datasets including millions of elements it becomes important to optimise searches. Trading off accuracy for speed can bring substantial performance improvements.

3.3 Hierarchical Navigable Small Worlds (HNSW) Algorithm

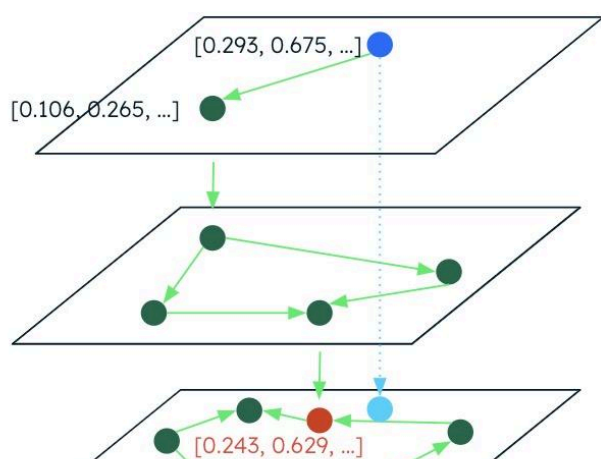
In this subsection, you'll learn about the HNSW algorithm employed by Atlas Vector Search, understand its importance in vector search, and explore its applications in powering semantic search and enhancing large language models. It reduces the overall search time, especially for large datasets. The HNSW algorithm enables various applications in AI, such as recommender systems, image and text retrieval, and anomaly detection.

3.3.1 What is the HNSW Algorithm?

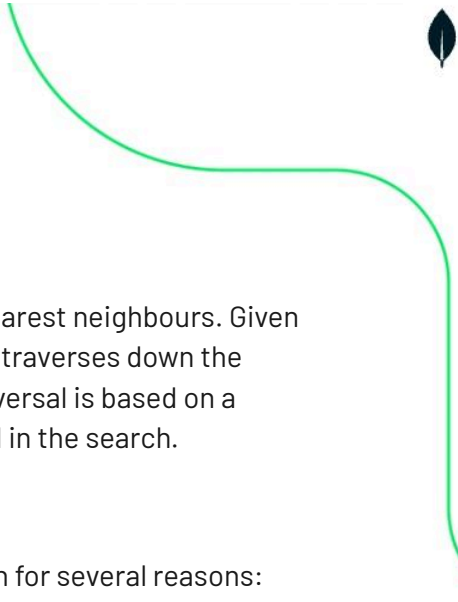
The Hierarchical Navigable Small Worlds (HNSW) algorithm is an efficient and scalable approach for approximate nearest neighbour (ANN) search in high-dimensional spaces. It was introduced by Yu. A. Malkov and D. A. Yashunin in 2016.

HNSW leverages vectors to build a hierarchical structure that allows for efficient nearest neighbour searches. It constructs a hierarchical graph, where each node represents a vector and edges represent the similarity between vectors.

The key idea behind HNSW is to divide the space of vectors into smaller regions or layers, creating a hierarchical structure. On the lowest layer, all vectors are interconnected, forming a dense graph. As we move up the hierarchy, vectors become sparser and more distant from each other.



During the construction phase of the HNSW algorithm, it starts by randomly selecting a vector to be the first node in the graph. Then, it iteratively adds new vectors to the graph, connecting them to existing nodes based on their similarity. The similarity between vectors



is calculated using a similarity function (Euclidean, Cosine, Dot Product).

Once the graph is constructed, it provides an efficient way to search for nearest neighbours. Given a query vector, HNSW starts at the top layer of the hierarchy and gradually traverses down the layers, moving towards vectors that are more similar to the query. This traversal is based on a navigation strategy called "pruning", where less similar vectors are skipped in the search.

3.3.2 Why is HNSW important in vector search?

HNSW, or Hierarchical Navigable Small World, is important in vector search for several reasons:

1. **Efficient Search:** HNSW is designed to perform fast approximate nearest neighbour search. It organises high-dimensional vectors into a hierarchical graph structure, which allows for efficient search operations by reducing the number of vectors that need to be examined.
2. **High Recall:** HNSW aims to achieve a high recall rate, meaning that it can find the most similar vectors with high accuracy.
3. **Approximate Search:** HNSW offers approximate search capabilities, meaning that it sacrifices a small degree of accuracy in favour of significantly faster search times. This is particularly useful in scenarios where real-time responses are required, such as recommendation systems or online search engines.
4. **Versatility:** HNSW can be applied to a wide range of applications that require vector search, such as image and video retrieval, text document search, recommendation systems, and anomaly detection.

Overall, HNSW is an important algorithm in vector search as it provides efficient, scalable, and accurate approximate nearest neighbour search, making it a valuable tool in various AI applications.

Atlas Vector Search uses the Hierarchical Navigable Small Worlds (HNSW) algorithm to perform searches. This allows Atlas Vector Search to find the most similar documents to a given query document, even if the documents are not in the same order in the database. This makes it ideal for semantic search tasks, such as finding the most similar documents to a given query document.



3.4 Text Embeddings and Chunking Techniques

3.4.1 Text Embeddings for Semantic Search

In this subsection, you'll understand the concept of text embeddings, their role in semantic search, and how they can be used to power vector search use cases.

We discussed before what dense vectors are so let's clarify a couple of things.

- Text embeddings and dense vectors are closely related concepts, and in many contexts, they are used interchangeably.
- Both refer to numerical representations of text (such as words or documents) that capture semantic meaning and context. In practice, when people talk about "text embeddings," they often refer to dense vectors, especially word embeddings, as you'll see later in the section.

In this subsection, you'll understand the concept of text embeddings, their role in semantic search, and how they can be used to power vector search use cases.

3.4.1.1 Introduction to Text Embeddings

Text embeddings, also known as word embeddings or word vectors, are numerical representations of words or text documents that are used in natural language processing (NLP) and machine learning. These representations capture the semantic and contextual information of words or text or any media, allowing machines to work with and understand human language more effectively.

The main idea behind text embeddings is to represent words or phrases in a multi-dimensional space, where semantically similar words or phrases are closer together. This allows us to perform mathematical operations on these vectors, such as calculating the similarity between two words or finding the most similar words to a given word.

There are several types of text embeddings, each with its own strengths and weaknesses. Some of the most popular ones include:

3.4.1.1.1 Word2Vec

Google's Word2Vec uses a shallow neural network to learn the vector representations of words in a large corpus of text.

3.4.1.1.2 GloVe

Short for Global Vectors for Word Representation, GloVe is an unsupervised learning algorithm developed by Stanford University. It combines the benefits of both global matrix factorization and local context window methods to generate word embeddings.



3.4.1.1.3 FastText

Facebook's FastText is an extension of Word2Vec that can generate embeddings for words and subwords. This allows it to handle out-of-vocabulary words and morphologically rich languages more effectively.

3.4.1.1.4 BERT

Bidirectional Encoder Representations from Transformers (BERT) is a pre-trained deep learning model developed by Google. It generates contextualised word embeddings by considering the context of a word within a sentence, making it more powerful than traditional word embeddings.

3.4.1.2 Semantic Search and Text Embeddings

Semantic search aims to improve search accuracy by understanding the contextual meaning of the query. Text embeddings play a crucial role in achieving this goal, as they allow us to represent and compare the meaning of words, phrases, media and even entire documents in a mathematical way.

By using text embeddings, we can perform tasks such as:

- Finding the most relevant documents for a given query
- Identifying similar documents or sentences
- Clustering documents based on their semantic content
- Detecting and correcting spelling errors in queries

3.4.1.3 Vector Embedding Indexing

Indexing in the context of vector embeddings is a method of organising data to optimise its retrieval and/or the storage. It's similar to the concept in traditional database systems, where indexing allows quicker access to data records. For vector embeddings, indexing aims to structure the vectors – roughly speaking – so that similar vectors are stored next to each other, enabling fast proximity or similarity searches.

3.4.1.4 Practical Applications and Challenges

Text embeddings have a wide range of applications in natural language processing, information retrieval, and machine learning. Some of the most common use cases include:

- Search engines: Improving the relevance of search results by understanding the semantic meaning of queries and documents.
- Recommendation systems: Suggesting similar items or content based on the semantic similarity of their descriptions.
- Sentiment analysis: Determining the sentiment of a piece of text by analysing the semantic meaning of its words and phrases.
- Machine translation: Translating text from one language to another by mapping words and phrases to their corresponding embeddings in the target language.



Despite their many advantages, text embeddings also come with some challenges:

- Computational complexity: Generating high-quality embeddings can be computationally expensive, especially for large vocabularies and deep learning models like BERT.
- Handling ambiguity: Words with multiple meanings can be challenging to represent accurately in a single vector space.
- Bias: Text embeddings can inherit biases present in the training data, which can lead to unintended consequences when used in downstream applications.

In order to understand better, let's see what embeddings look like. Use the curl command to generate embeddings for the input text. Try to change the value of the input text to create a different vector, and paste your output:

Retrieve embeddings for text

```
curl --request POST \  
  --url https://api.voyageai.com/v1/embeddings \  
  --header "Authorization: Bearer $VOYAGE_AI_KEY" \  
  --header "content-type: application/json" \  
  --data '  
{  
  "input": ["Sample Document goes here"],  
  "model": "voyage-4-large"  
}  
'
```

Your Input: *<your-input-text>*
Corresponding Embeddings: *<paste-embeddings-here>*

In conclusion, text embeddings are a powerful tool for representing and processing natural language data. By understanding their strengths, weaknesses, and applications, you can harness their potential to build more accurate and efficient semantic search systems and enhance large language models.

3.4.2 Chunking Techniques

In this subsection, we address the concept of chunking, its importance, and how to implement it.

3.4.2.1 What is Chunking?

Chunking is a computational method used in the field of artificial intelligence and natural language processing. It involves the process of dividing a sequence of words or tokens into meaningful units, known as chunks. Chunks typically consist of a group of words that form a syntactically coherent and semantically important unit.

In natural language processing, chunking is used to identify and extract specific parts of speech or phrases from a sentence. This can include noun phrases, verb phrases, prepositional phrases, and more. By breaking down a sentence into chunks, it becomes easier to analyse and understand the underlying structure and meaning of the text.

The process of chunking usually involves the use of techniques such as part-of-speech tagging, which assigns a specific grammatical label to each word in a sentence. Then, using predefined rules or patterns, these labelled words are combined into chunks based on their syntactic relationships.

For example, consider the sentence: "The cat sat on the mat." Through chunking, the noun phrase "The cat" and the prepositional phrase "on the mat" can be identified as separate chunks. This chunking process helps identify the subject and the location specified in the sentence.

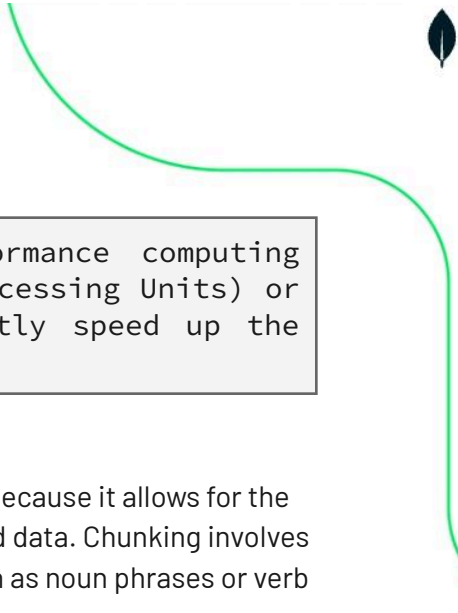
Dividing the data into smaller pieces can also lead to the loss of important contextual information. It may hinder the model's ability to capture the relationships and dependencies between different parts of the data.

On the other hand, while processing larger chunks of data can provide more context and potentially capture more intricate patterns, it requires a higher computational power and can be more resource-intensive.

Level of chunking also determines whether the model will lose context or not, i.e. its failure to accurately understand and generate text within the appropriate context, leading to inaccurate or nonsensical responses.

In general, computing embeddings can be computationally expensive for large datasets. For simple embedding algorithms and small datasets, a regular desktop or laptop computer with a modern processor and a decent amount of RAM (e.g., 8GB or more) should be sufficient.

However, for more complex algorithms or larger datasets, more



powerful hardware may be required. High-performance computing systems, such as servers with GPUs (Graphics Processing Units) or TPUs (Tensor Processing Units), can significantly speed up the computation time for embedding generation.

3.4.2.2 Why is Chunking Important?

Chunking is an important concept in the field of Artificial Intelligence (AI) because it allows for the interpretation and extraction of meaningful information from unstructured data. Chunking involves dividing a sequence of words or data into meaningful units or chunks, such as noun phrases or verb phrases.

1. Parsing text: Chunking helps in parsing and understanding the structure of sentences or texts. By identifying and grouping together related words, chunking assists in extracting information and identifying the syntactic relationships between words.
2. Named entity recognition: Chunking is used in named entity recognition, which involves identifying and classifying specific types of entities in text, such as people, organisations, locations, or dates. By chunking together consecutive words that refer to the same entity, named entities can be accurately identified and categorised.
3. Information extraction: Chunking plays a crucial role in information extraction from unstructured data sources. By identifying and extracting meaningful chunks of information, AI systems can analyse and understand the content more effectively, enabling tasks like sentiment analysis, summarization, or question-answering.
4. Machine learning and natural language processing: Chunking serves as a pre-processing step in many AI techniques, such as machine learning or natural language processing. It helps in transforming raw text data into structured representations that can be further analysed or processed by AI models.

In summary, chunking is important in AI because it enables the interpretation, extraction, and understanding of meaningful information from unstructured data, contributing to various applications like text parsing, named entity recognition, information extraction, and natural language processing.

3.4.2.3 Exercise

Now that we know the importance of chunking and text embedding, let's learn how to implement it from a simple example:

Create a new file in your project directory called **chunking.py**,

and add the following content:

```
import os
import requests
import json
import voyageai

MY_INPUT_FOR_CHUNKS_AND_EMBED = (
    "Hey, trying this long form of input. Has to be pretty long. Is this enough?"
)
split_array = MY_INPUT_FOR_CHUNKS_AND_EMBED.split(".")
VOYAGE_AI_KEY = os.environ["VOYAGE_AI_KEY"]

if VOYAGE_AI_KEY is None:
    print("Please set the API key as an environmental variable in this terminal")

vo = voyageai.Client(api_key=VOYAGE_AI_KEY)
embeddings = vo.embed(
    split_array, model="voyage-4-large", input_type="document"
).embeddings

for index, embedding in enumerate(embeddings):
    print("The subtext below has been vector embedded as:")
    print(split_array[index])
    print(embedding)
```

That's it! You have now created a script that implements the chunking of a multi-sentence string by splitting it on a delimiter (period "." in this case).

Run the script with the following command:

```
python3 chunking.py
```

When you run the script, it will split the input string into sentences and print corresponding embeddings. For the sample input in the script, you should get the following output:

```
The subtext below has been vector embedded as:
Hey, trying this long form of input
[
  -0.013134486, -0.0033105228, ...
  ... 1436 more items
```

```
]
The subtext below has been vector embedded as:
Has to be pretty long
[
    -0.025483789,  0.0026804176,    ...
...
... 1436 more items
]
The subtext below has been vector embedded as:
Is this enough?
[
    0.007243171,    0.016098801,    ...    ...
... 1436 more items
]
```

Change the input in the script, run it and paste your output in the box below:

3.4.3 Chunking Techniques

In this subsection, we'll learn different chunking techniques used in the creation of embeddings. By the end of this subsection, you'll have an understanding of various chunking methods, which will help you make informed decisions when working with text embeddings in your projects.

Chunks can be individual words, sentences, or even paragraphs. The goal of chunking is to extract meaningful information from text (and still have enough context to make the chunk meaningful for the purpose intended), which can then be used for various tasks like sentiment analysis, named entity recognition, or text classification.

Most common chunking techniques used in natural language processing are:

- **Word-level** chunking involves dividing text into individual words, focusing on identifying patterns and relationships between adjacent words. For example text summarization, machine translation, and sentiment analysis, etc.
- **Sentence-level** chunking involves identifying and grouping complete sentences, facilitating understanding and analysis of the overall meaning in a text. For example information extraction from unstructured text, such as news articles or research papers.

- **Paragraph-level** chunking extends this concept to larger sections of text, allowing for the identification and extraction of key information and themes. For example more accurate document classification or topic modelling by capturing the topical composition at a finer granularity, etc.
- **Recursive** chunking involves using a set of hierarchical separators (such as newlines, punctuation, and spaces) to split the document into evenly sized chunks. If the chunk is larger than the target size, the function will call itself using the next separator. This results in paragraphs, sentences, and words being kept together as much as possible.

The chunking strategy is important for the use case the AI is being used for. To add to the complexity, there are also static and dynamic chunking strategies.

To better your understanding, consider the following use cases and write the appropriate chunking techniques suitable for them in the adjacent column:

Use Case	Suitable Chunking Technique
Sentiment analysis	Word Level
Information extraction from news articles	
Categorising documents	
Text summarization	

3.4.3.1 Exercise: Chunk and Embed

Building more upon the previously created Python project to perform chunking, we'll create a new file called `chunk_and_embed.py` and use the AI MDB Azure server to get embeddings for different types of chunks, in similar fashion. We'll continue to use OpenAI's `text-embedding-ada-002` model as referenced below:

```
import requests
import os
import json
import voyageai

try:
    VOYAGE_AI_KEY = os.environ["VOYAGE_AI_KEY"]
except:
```

```

        raise KeyError("the key VOYAGE_AI_KEY was not found in the
environmental variables. Please set it to continue.")

def chunk_text(text, chunk_size, chunk_type = "word"):
    chunks = []
    if chunk_type == "word":
        words = text.split(" ")
        for i in range(0, len(words), chunk_size):
            chunk = " ".join(words[i : i + chunk_size])
            chunks.append(chunk)
    elif chunk_type == "sentence":
        sentences = text.split(".")
        for i in range(0, len(sentences), chunk_size):
            chunk = ". ".join(sentences[i : i + chunk_size])
            chunks.append(chunk)
    elif chunk_type == "paragraph":
        paragraphs = text.split("\n")
        for i in range(0, len(paragraphs), chunk_size):
            chunk = "\n".join(paragraphs[i : i + chunk_size])
            chunks.append(chunk)

    # Remove any empty chunks
    chunks = [chunk for chunk in chunks if chunk and not
chunk.isspace()]
    return chunks

def recursive_chunking(text, separators= ["paragraph", "sentence",
"word"], max_characters=150, level=0):

    if len(text) <= max_characters:
        return [text]

    if level > len(separators):
        return [text]

    good_chunks = []
    chunks = chunk_text(text, chunk_size=1,
chunk_type=separators[level])
    for chunk in chunks:
        if chunk.strip() == '':
            continue

        if len(chunk) < max_characters:
            good_chunks.append(chunk)

```

```

        else:
            sub_chunks = recursive_chunking(chunk,
separators=separators, level=level+1)
            good_chunks.extend(sub_chunks)

    good_chunks = [chunk for chunk in good_chunks if chunk or not
chunk.isspace()]
    return good_chunks

def get_embeddings(search_input):

    vo = voyageai.Client(api_key=VOYAGE_AI_KEY)

    try:
        embeddings = vo.embed([search_input],
model='voyage-3-large').embeddings
        return embeddings[0]
    except requests.exceptions.RequestException as e:
        print(f"An error occurred: {e}")
        return None

def get_chunk_embeddings(chunks):

    embeddings = []
    for chunk in chunks:
        response = get_embeddings(chunk)
        embeddings.append(response)
    return embeddings

```

```

if __name__ == "__main__":
    text = ""

```

The girl woke early. She dressed. She packed her bag. She left quietly. The morning was cold. Her breath made clouds. Her boots crunched frost. Trees stood still. The sky was pale. A crow cawed. She didn't look back. She couldn't. She wouldn't.

Hours passed beneath a shifting sky as she followed the winding trail, her thoughts blending with the rustling of the wind through bare branches and the distant, rhythmic sound of a river carving its way through rock and memory. Every step she took seemed to echo with questions she wasn't ready to answer and feelings she wasn't ready to name, but she pressed forward with a kind of

stubborn grace, drawn by a need she didn't fully understand. She remembered stories her grandfather used to tell about the hill beyond the forest, where the sun touched the earth and wishes—if spoken aloud and true—could linger long enough to find a home in the world.

When she finally reached the clearing, the sun had dipped low and gold spilled over the grass like warm honey. She stood in the silence, heart racing, breath catching, then whispered her wish into the hush—and the wind, gentle now, carried it away.

"""

```
chunk_size = 3

word_chunks = chunk_text(text, chunk_size, "word")
word_embeddings = get_chunk_embeddings(word_chunks)

sentence_chunks = chunk_text(text, 1, "sentence")
sentence_embeddings = get_chunk_embeddings(sentence_chunks)

paragraph_chunks = chunk_text(text, 1, "paragraph")
paragraph_embeddings = get_chunk_embeddings(paragraph_chunks)

recursive_chunks = recursive_chunking(text, max_characters=50)
recursive_embeddings = get_chunk_embeddings(recursive_chunks)

print("Word-level chunks:", word_chunks)
print('Word-level embeddings:', word_embeddings[0][:10])

print("Sentence-level chunks:", sentence_chunks)
print('Sentence-level embeddings:',
sentence_embeddings[0][:10])

print("Paragraph-level chunks:", paragraph_chunks)
print('Paragraph-level embeddings:',
paragraph_embeddings[0][:10])

print("Recursive-level chunks:", recursive_chunks)
print('Recursive-level embeddings:',
recursive_embeddings[0][:10])
```

Run the script with the following command:

```
python3 chunk_and_embed.py
```

If you have time, you can change the input text and experiment with changing the "max_characters" parameter in the recursive_chunking() call to see how different texts get handled.

Note that the runtime of this example increases at a “geometric rate” as you increase the size of the input text.

Change the chunkSize from 3 to 5, run the script again and paste your output’s first 10 array items here:

Word-level Chunks	
Word-level Embeddings	
Sentence-level Chunks	
Sentence-level Embeddings	
Paragraph-level Chunks	
Paragraph-level Embeddings	
Recursive-level Chunks	
Recursive-level Embeddings	



3.5 Using Atlas Vector Search

3.5.1 Building Movie Recommendation app

Let's use what we've learned so far in this course, to create a practical solution. We'll build a movie recommendation application that allows users to find movies based on similar storylines/plots. The application should allow users to type what kind of story they are interested in, and respond with the movies having the story closest to that, i.e. belonging to the similar storylines based on the semantic context of the user input.

We'll use Atlas Search's Vector Search functionality to power the search use case, instead of Atlas Search's synonym based search because the former helps improve the semantic relevance of search results, making it easier for users to find the movies they should watch based on context and meaning. It also removes the need to manually create the synonym collection with a fixed set of words and their synonyms.

We'll use the Atlas cluster created in the Setup stage of the course. From the sample data set loaded in the cluster, we'll be using the `sample_mflix.embedded_movies` namespace. The `embedded_movies` collection already stores embeddings in the `plot_embeddings` field, created using the `plot` field.

3.5.2 Create Atlas Vector Search indexes

Go to your Atlas cluster page and create the following Atlas Vector Search Indexes using the JSON Editor.

MongoDB Atlas

ORGANIZATION > PROJECT > CLUSTER Cluster0

[Back to Search Indexes](#)

Create a Vector Search Index

1 Setup 2 Configuration 3 Review

Start Your Index Configuration

Search Type
Which Search type should I use?

Atlas Search
Full-text search for relevance-based app features.

Vector Search
For semantic search and AI applications.

Index Name and Data Source
Search indexes are specific to a database and collection.

At this time, search indexes cannot be created for time series collections.

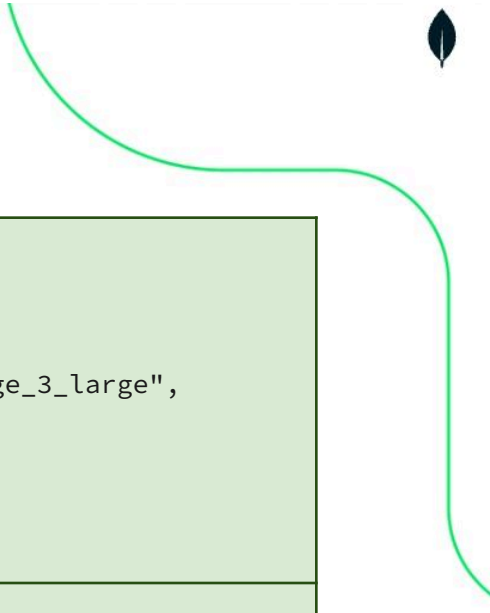
Index Name
cosineidx

Database and Collection
Search for database or collection ...

- langchain_demo
- sample_airbnb
- sample_analytics
- sample_geospatial
- sample_guides
- sample_mflix
 - comments
 - embedded_movies**
 - movies
 - sessions

Create 3 different vector search indexes in your Atlas cluster for sample_mflix.embedded_movies.

Use the first column in the table below as the index name and the second as the description:



<i>cosineIdx</i>	<pre>{ "fields": [{ "type": "vector", "path": "plot_embedding_voyage_3_large", "numDimensions": 2048, "similarity": "cosine" }] }</pre>
<i>euclideanIdx</i>	<pre>{ "fields": [{ "type": "vector", "path": "plot_embedding_voyage_3_large", "numDimensions": 2048, "similarity": "euclidean" }] }</pre>
<i>dotProductIdx</i>	<pre>{ "fields": [{ "type": "vector", "path": "plot_embedding_voyage_3_large", "numDimensions": 2048, "similarity": "dotProduct" }] }</pre>

Following are the key features of each document in the fields array of this index mapping definition:

- **type**
vector is the type assigned to the field storing the embeddings, for vector search.
- **numDimensions**
The numDimensions field tells the vector search index how many dimensions each vector in your dataset has. It enables the index to understand the structure and shape of the vectors. By specifying the correct number of dimensions, you ensure that the index can accurately perform vector-based operations, such as similarity search or clustering.

It is important to note that the number of dimensions is a fixed value based on the characteristics of your dataset. If you incorrectly specify the dimensions, it could lead to inaccurate search results

or errors.

- **similarity**

Atlas Search's vector search supports various types of similarities, namely:

- **euclidean:** It measures the distance between two data points in a multi-dimensional space using the Euclidean distance formula. The *smaller* the Euclidean distance, the more similar the points are considered to be.
- **cosine:** It measures the similarity between two vectors by calculating the cosine of the angle between them. It disregards the magnitude of the vectors and focuses on the orientation. A *higher* cosine similarity value indicates a greater similarity between the vectors.
- **dotProduct:** It measures the similarity between two vectors by calculating the dot product of the vectors. It captures both the magnitude and the orientation of the vectors. A *higher* dot product value indicates a greater similarity between the vectors.

3.5.3 Perform search

3.5.3.1 Preparing a Python application for searching

Let's create our server which will allow us to:

- Initialise MongoDB client
- Performs vector search and return the response

To achieve this, create a new file called `search.py` in the project, with the following content:

```
from pymongo import MongoClient
from flask import Flask, request, jsonify
import requests
import json
import os
import voyageai

# Enter your MongoDB connection URI and collection name here
MONGO_URI =
'mongodb+srv://<user>:<password>@atlascluster.<your_cluster>.mongo
db.net/?retryWrites=true&w=majority'
VOYAGE_AI_KEY = os.environ["VOYAGE_AI_KEY"]
MONGODB_COLLECTION = "embedded_movies"
MONGODB_DATABASE = "sample_mflix"
```

```

client = MongoClient(MONGO_URI)
port = 3000
app = Flask(__name__)

# This function uses the endpoint to fetch embeddings
def get_embeddings(search_input):

    vo = voyageai.Client(api_key=VOYAGE_AI_KEY)

    try:
        embeddings = vo.embed([search_input],
model='voyage-3-large', output_dimension=2048).embeddings
        return embeddings[0]
    except requests.exceptions.RequestException as e:
        print(f"An error occurred: {e}")
        return None

# Function to perform Atlas Vector search on embedded_movies
def atlas_search(user_input, index_name):
    try:
        if (
            not user_input
            or not index_name
            or user_input.strip() == ""
            or index_name.strip() == ""
        ):
            print("user_input", user_input)
            print("index_name", index_name)
            raise ValueError("Invalid inputs")
        collection = client[MONGODB_DATABASE][MONGODB_COLLECTION]

        results = collection.aggregate(
            [
                {
                    "$vectorSearch": {
                        "index": index_name,
                        "path": "plot_embedding_voyage_3_large",
                        "queryVector": get_embeddings(user_input),
                        "numCandidates": 100,
                        "limit": 5,
                    }
                },
                {

```

```

        "$project": {
            "_id": 0,
            "title": 1,
            "score": {"$meta": "vectorSearchScore"},
        },
    ],
    ).to_list()
    return results
except Exception as e:
    print("Error performing Atlas Vector search")
    raise e

@app.route("/search")
def search():
    try:
        response = atlas_search(
            request.args.get("userInput"),
            request.args.get("idxName")
        )
        return jsonify(response)
    except Exception as e:
        print(e)
        return "Internal Server Error", 500

if __name__ == "__main__":
    print(f"Server listening at http://localhost:{port}")
    app.run(debug=True, port=port)

```

Replace the database connection string defined in the variable `MONGODB_URI` with your Atlas cluster's connection string.

Start the server using the following command:

```
python3 search.py
```

The following message should appear in the terminal:

```
Server listening at http://localhost:3000
```

3.5.3.2 The Vector Search Query

The vector search query used here is as follows:

```
[
  {
    "$vectorSearch": {
      "exact": "false",
      "index": "default",
      "path": "plot_embedding_voyage_3_large",
      "queryVector": [
        0.43,
        1.2,
        3.4,
        4.3
        // . . . other values . . .
      ],
      "numCandidates": 50,
      "limit": 5
    }
  },
  {
    "$project": {
      "_id": 0,
      "title": 1,
      "score": {
        "$meta": "vectorSearchScore"
      }
    }
  }
]
```

\$vectorSearch is the name of the stage.

- **exact** is the flag that specifies the search algorithm (ANN if false, or ENN if true). This parameter can be omitted (default is false).
- **index** is the name of the index created in [subsection 3.5.2](#)
- **path** is the field name in the collection to search on
- **queryVector** is the vector created from translating user's plain text input into embeddings using AI MongoDB Azure Endpoint (OpenAI's *text-embedding-ada-002*)
- **numCandidates** specifies the number of nearest neighbours to use during the search. Value must be less than or equal to ($\leq$) 10000. Recommended ratio of ten to twenty nearest neighbours for a *limit* of one document.
- **limit** specifies the number of documents to return in the results. ($limit \leq numCandidates$). Between 10-20 candidates for 1 limit ratio is recommended.

1. The search query here fetches results using the similarity type which is defined in the mapping of the search index being passed as the query parameter
2. **numCandidates** parameter allows increasing accuracy with higher values although might impact latency.

3.5.3.2 Exercise

Use the following command to search for the recommended movies:

```
curl --get --data-urlencode "userInput=<user-input>"  
--data-urlencode "idxName=<index-name>"  
http://localhost:3000/search
```

Here:

- **userInput**: Accepts the user input for recommendation
- **idxName**: The index (out of the three created in the previous step) to be used for search.

Acceptable values are:

- *cosineIdx*
- *euclideanIdx*
- *dotProductIdx*

Example:

```
curl --get --data-urlencode "userInput=Comedy movie with fighting  
scenes" --data-urlencode "idxName=cosineIdx"  
http://localhost:3000/search
```

Example of the output:

```
[  
  {  
    "score": 0.9352330565452576,  
    "title": "Jesus Christ Vampire Hunter"  
  },  
  {  
    "score": 0.9284485578536987,  
    "title": "Fighting"  
  },  
  {  
    "score": 0.9281926155090332,  
    "title": "Shanghai Express"  
  },  
  {  
    "score": 0.927066445350647,
```



```

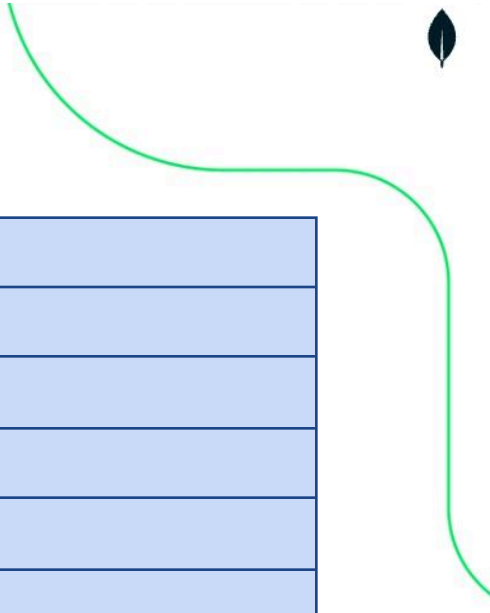
    "title": "Project A"
  },
  {
    "score": 0.9269802570343018,
    "title": "Never Back Down"
  }
]

```

Run the following user inputs, one at a time, and paste your results in the table below.

You will notice that you always get the same titles, but the scores will change depending on which index you use. This is because in the vector search space, there aren't enough entries to cause a substantial difference in the order of the results. Nevertheless, it is recommended that you use the same indexing model for search as was used to train the model, which in this case was 'cosine' similarity.

User Input	Titles of the top 5 movies in result with different similarities		
	cosineIdx	euclideanIdx	dotProductIdx
Comedy movie with fighting scenes			
Travelling movie set in historical period			
Violence and			



	Thriller			
	Romantic feel good			

3.5.4 Perform Image search with different embedding

3.5.4.1 Image search using a different embedding

In this scenario we will use a collection of product images (small sample) from an apparel company, and we will use plain text to perform searches in the product catalog using just descriptions in english. For instance: "pink shoes", "green t-shirts", "nike sportswear", "yellow", "handbags", etc.

We will use a new collection of "products" that contain product images and embeddings of those images.

- Download the Products collection, unzip the dump folder and restore it in your Atlas cluster (Note: you need at least 320KB to upload this collection)
- Load the collection into your Atlas
- Create the corresponding index for this collection using filters for price, averageRating, and discountPercentage.

This collection has been stored as a binary file with mongodump. This is inside the /root/da660/mongodb-atlas-vector-search-image-products/encoder folder. Unzip this file and then restore the collection in your Atlas cluster with mongorestore:

```
cd /root/da660/mongodb-atlas-vector-search-image-products/encoder
```

```
unzip dump.zip
```

```
mongorestore
```

```
--uri="mongodb+srv://<user>:<password>@<your_atlas_cluster>.mongodb.net/?authSource=admin" ./dump/vector_search/products.bson
```

The output should be as follows:

```
checking for collection data in dump/vector_search/products.bson
```

```
reading metadata for vector_search.products from  
dump/vector_search/products.metadata.json
```

```
restoring vector_search.products from  
dump/vector_search/products.bson
```

```
finished restoring vector_search.products (42 documents, 0  
failures)
```

```
no indexes to restore for collection vector_search.products  
42 document(s) restored successfully. 0 document(s) failed to  
restore.
```

Review the collection you downloaded. You can inspect these new documents using the Browse Collections option in the Atlas UI.

Now you will need to create the collection index - In the Atlas Search tab, create the following Vector search index with Index name "vectorImg" for the "vector_search.products" Collection:

vectorImg

```
{  
  "fields": [  
    {  
      "numDimensions": 768,  
      "path": "imageVector",  
      "similarity": "cosine",  
      "type": "vector"  
    },  
    {  
      "path": "price",  
      "type": "filter"  
    },  
    {  
      "path": "averageRating",  
      "type": "filter"  
    }  
  ]  
}
```

```
    },  
    {  
      "path": "discountPercentage",  
      "type": "filter"  
    }  
  ]  
}
```

Review the encoder/ folder and make sure the images/ folder includes the image files. These were the original images that were encoded into embeddings to create the products collection.

In this exercise we will use a Python script to create an application to allow this search in the product collection. Make sure you are in the folder `mongodb-atlas-vector-search-image-products`

```
cd ~/da660/mongodb-atlas-vector-search-image-products
```

Using the IDE (or vim or nano in the terminal), open `config/config_database.py` and change the `mongo_uri` path to point at your Atlas cluster.

```
mongo_uri =  
"mongodb+srv://<user>:<password>@<atlas_cluster>/?retryWrites=true&w=ma  
jority"  
db = "vector_search"  
collection = "products"
```

(Optional) - This example uses an LLM that maps images and text into a shared vector space. The LLM is available in the Virtual Lab. To use the local copy, make a single change to `webapp/flask_server.py` line 19:

```
Change  
  
preTrainedModelName = "clip-ViT-L-14"  
  
to  
  
preTrainedModelName = "/root/da660/model/clip-ViT-L-14"
```

If you don't make the above change, Python will download the model at runtime.

Now the code is ready to run. You might need to click on the refresh button if using the VSCode IDE to see the new directory. Go to the webapp/ folder and run flask_server.py:

```
cd webapp/  
  
python3 flask_server.py
```

The output should be as follows:

```
* Serving Flask app 'flask_server' (lazy loading)  
* Environment: production  
  WARNING: This is a development server. Do not use it in a  
  production deployment.  
  Use a production WSGI server instead.  
* Debug mode: on  
WARNING: This is a development server. Do not use it in a  
production deployment. Use a production WSGI server instead.  
* Running on http://localhost:5010  
Press CTRL+C to quit  
* Restarting with stat  
* Debugger is active!
```

If you are working locally, open the browser to launch the web app on <http://localhost:5010>

If you are using the Instruqt lab, use the browser tab to launch this URL.

Then use the search box to test this by looking for “pink”, and check the results.



Demo: MongoDB Atlas Vector Search on Product Images

Product:

Search Results

10 results were found for the search for pink



Image File: ./images/33.jpg

- The data in the database:
- Price: \$36.3
- Average Rating: 4.01
- Discount Percentage: 18%
- Search score: undefined
- Embeddings (only 5 dimensions displayed):
[0.6016309261322021,0.21600817143917084,0.7800920605659485,0.28897297382354736,0.12988612055778503...]



Image File: ./images/36.jpg

- The data in the database:
- Price: \$62.11
- Average Rating: 4.61
- Discount Percentage: 16%
- Search score: undefined
- Embeddings (only 5 dimensions displayed):
[0.973077654838562,-0.07471618056297302,0.4661506414413452,1.5796294212341309,-0.4271528720855713...]

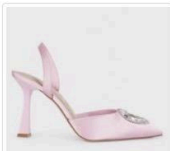


Image File: ./images/32.jpg

- The data in the database:
- Price: \$36.43
- Average Rating: 4.08
- Discount Percentage: 21%
- Search score: undefined
- Embeddings (only 5 dimensions displayed):
[0.31679779291152954,0.3058982789516449,-0.03351714462041855,1.2151589393615723,0.07513676583766937...]

You can try other terms like “pink shoes”, “sportswear”, “nike sportswear”, etc. You can verify by checking the vector_search.products namespace.

Let's now switch to the advanced version that uses the filters we defined in the vector search index. Go to launch the web app on <http://localhost:5010/advanced> and modify the filters to set a cap on Price, minimum discount and rating values.

For example:

Product:	pink
Maximum Price (\$0-\$100):	90
Minimum Rating (1-5):	3
Minimum Discount (%):	5
Show in the Top:	☐ More Relevant ☐ Higher Average Rating ☒ Cheaper
Search	

Review the results of the search from the list. Modify your search, filters, or sort order.
You can review the code in the flask_server.py to see how these were applied in the pipeline:

```
vector_query = model.encode(vector_text).tolist()
pipeline = [
    { "$vectorSearch": {
        "index": "vectorImg",
        "path": "imageVector",
        "queryVector": vector_query,
        "numCandidates": 150,
        "limit": 10,
        "filter": {
            "$and": [
                {
                    "price": {
                        "$lt": maximumPrice
                    }
                },
                {
                    "averageRating": {
                        "$gte": minRating
                    }
                },
                {
                    "discountPercentage": {
                        "$gte": minDiscount
                    }
                }
            ]
        }
    }
},
    { "$project": {
        "imageVector": {"$slice": ["$imageVector", 5]},
        "imageFile": 1,
        "price": 1,
        "discountPercentage": 1,
        "averageRating" : 1,
        "_id": 0,
        'score': {
            '$meta': "vectorSearchScore"
        }
    }
},
    { "$sort" : sort
```

```
}  
]  
# Execute the pipeline  
docs = list(product_collection.aggregate(pipeline))
```

Conclusions:

- This exercise shows how accurate and flexible the user search is without creating any meta field, categorization, or tagging in the product collection.
- The images were encoded using the embedding model [clip-ViT-L-14](#). This is the Image & Text model [OpenAI CLIP](#), which maps text and images to a shared vector space. CLIP (Contrastive Language-Image Pretraining) model predicts the most relevant text snippet given an image. For applications of the models, have a look in [SBERT.net - Image Search](#)
- We used [Sentence Transformers Python framework](#) to encode the product images in the Python application (<https://sbert.net/>) and create a MongoDB collection with them using the **encoder/encoder_and_loader.py** file. Note: This was prepared before the course as encoding can take some time, so students just restore the collection in their cluster.
- We can create a vector search index with filters to perform advanced search with conditions when we use the \$vectorSearch with filters.
- The number of dimensions when defining the vector index was based on the number of array elements in the embeddings. The embedding model clip-ViT-L-14 returns 768 dimensional arrays.
- The similarity chosen in this case was cosine. This is, in general, the one recommended when searching images. The model was trained to maximize the cosine similarity.
- Can you think of different usages you could apply this for? Here are some other examples:
 - [Audio find - Atlas Vector search for audio](#)
 - [Width.ai use case: Finding similar products with CLIP](#) (external)

Cleaning up:

Once you are done, you can kill the 'flask_server' app using the keyboard shortcut Ctrl+c

3.5.4.2 Challenge: Similar Image Search

Based on the previous section, using the Python code provided, can you evolve the code to create additional functionality in the web app as follows?

Instead of searching for products using plain text, users can provide an image of the product, and the vector search will return product images similar to the image provided.

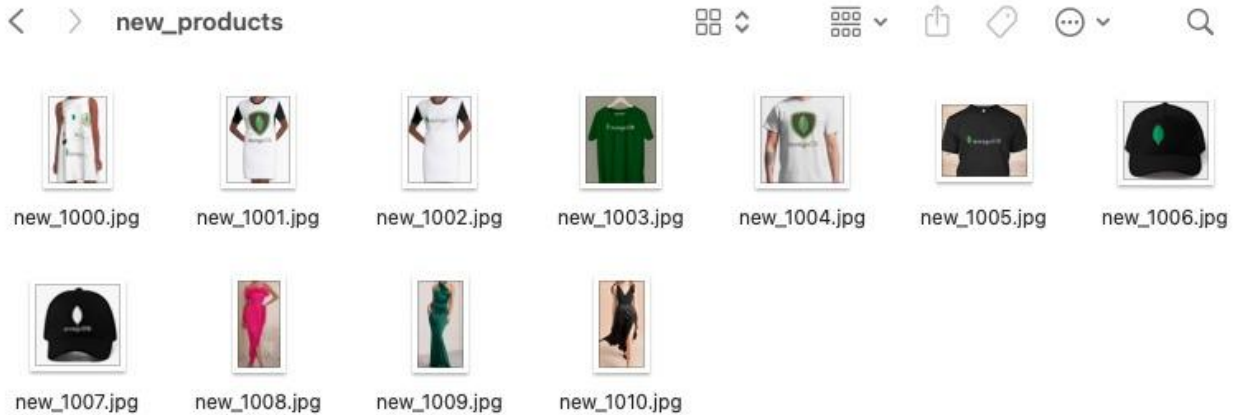
Create a new route in the web app to access it:

<http://localhost:5010/similar>

To test this you can use the new images provided in this AWS S3 bucket:

https://mdb-strigio.s3.eu-central-1.amazonaws.com/vectorsearch_new_products/<image_name>

Image names follow this naming convention: "new_1000.jpg", "new_1001.jpg", "new_1002.jpg", etc.



Using the IDE, under the webapp/templates folder, duplicate the index.html file and rename the copy as similar.html.

Then make sure the search_button click function calls the url: /searchSimilar as display in the image below:



Update the flask_server.py file, to add this functionality, by adding the new Route /searchSimilar and the new function:

```
from PIL import Image
import requests
from io import BytesIO
...
@app.route('/searchSimilar', methods=['GET'])
def searchSimilar():
    vector_img_path = request.args.get('vector', default=None,
    type=str)

    ...
```

```

img = Image.open(vector_img_path)

vector_img_query = model.encode(img).tolist()

pipeline = [
    {
        "$vectorSearch": {
            "index": "vectorImg",
            "path": "imageVector",
            "queryVector": vector_img_query,
            ...

```

You need to complete the above implementation so that it can do the following:

1. Determine whether the image link was relative or on the web
2. If it was an HTTP link, request the image and parse the response content with BytesIO so that `Image.open()` can read it.

Also create a route for the new URL `/similar` to render the `similar.html` template:

```

@app.route('/similar')
def similar():
    return render_template("similar.html")

```

(TIP: copy the `index.html` file)

Work with your instructor if you do not have experience with Python coding.

The code is ready now to be executed, create the Python environment:

```

//In the mongodb-atlas-vector-search-image-products dir

cd webapp/
python3 flask_server.py

```

Go to launch the web app on `http://localhost:5010/similar` and enter the path for the image, it can be local or from the internet (you can also use the S3 bucket to use some demo images)

For instance, enter this image path:

`https://mdb-strigio.s3.eu-central-1.amazonaws.com/vectorsearch_new_products/new_1008.jpg`



Demo: MongoDB Atlas Vector Search on Product Images

Image
Path
(Global
access):

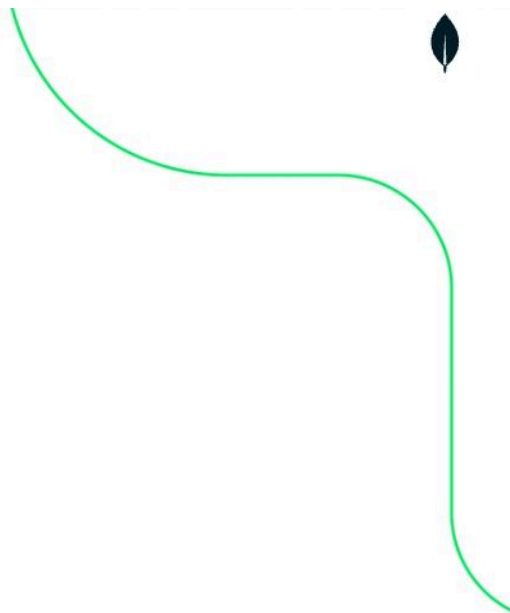
Search Results

3 results were found for the search for `https://mdb-strigio.s3.eu-central-1.amazonaws.com/vectorsearch_new_products/new_1008.jpg`



Image File: `/images/29.jpg`

- The data in the database:
- Price: \$7.23
- Average Rating: 3.01
- Discount Percentage: 19%
- Search score: 0.8556597232818604
- Embeddings (only 5 dimensions displayed):
[0.35963475704193115, 0.05992698669433594, 0.43685877323150635, 0.11992008984088898, 0.11184409260749817...]



Note you can also search for your local images, for instance,;: ../encoder/images/30.jpg

Image Path (Global access):

Search Results

3 results were found for the search for ../encoder/images/30.jpg



Image File: ../images/30.jpg

- The data in the database:
- Price: \$49.71
- Average Rating: 3.99
- Discount Percentage: 19%
- Search score: 1
- Embeddings (only 5 dimensions displayed):
[0.3709647059440613,0.11045925319194794,0.29688388109207153,0.5012639760971069,0.06695768237113953...]

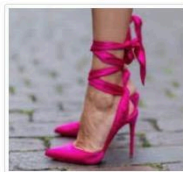


Image File: ../images/33.jpg

- The data in the database:
- Price: \$36.3
- Average Rating: 4.01
- Discount Percentage: 18%
- Search score: 0.939346194267273
- Embeddings (only 5 dimensions displayed):
[0.6016309261322021,0.21600817143917084,0.7800920605659485,0.28897297382354736,0.12988612055778503...]

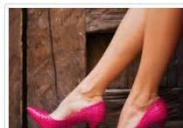


Image File: ../images/29.jpg

- The data in the database:
- Price: \$7.23
- Average Rating: 3.01
- Discount Percentage: 19%
- Search score: 0.9312969446182251
- Embeddings (only 5 dimensions displayed):
[0.35963475704193115,0.05992698669433594,0.43685877323150635,0.11992008984088898,0.11184409260749817...]

../encoder/images/2.jpg

Image Path (Global access):

../encoder/images/2.jpg

Search

Search Results

3 results were found for the search for ../encoder/images/2.jpg



Image File: ./images/2.jpg

- The data in the database:
- Price: \$58.34
- Average Rating: 3.82
- Discount Percentage: 9%
- Search score: 1
- Embeddings (only 5 dimensions displayed):
[0.15310931205749512,0.8425386548042297,-0.22332631051540375,1.050410270690918,-0.04842076450586319...]



Image File: ./images/4.jpg

- The data in the database:
- Price: \$93.8
- Average Rating: 3.5
- Discount Percentage: 8%
- Search score: 0.9509576559066772
- Embeddings (only 5 dimensions displayed):
[0.4768178164958954,0.9435704946517944,-0.17211812734603882,0.860195517539978,0.04907296597957611...]

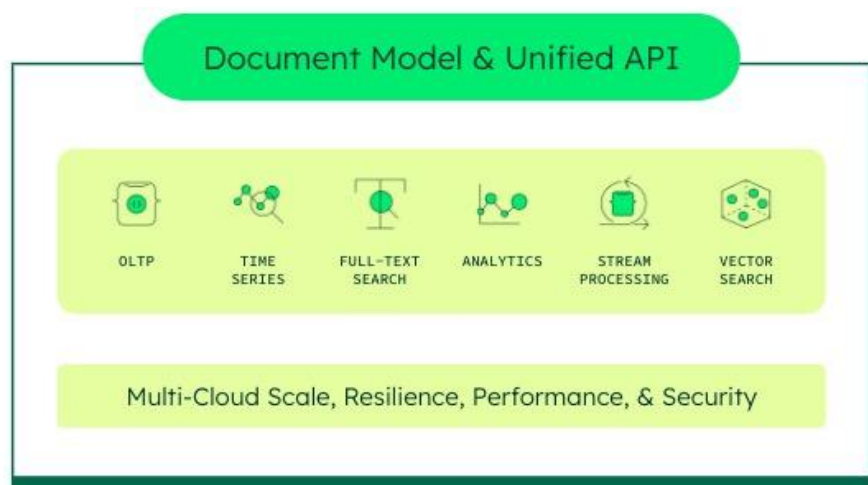
Try any images on the internet, check the image in this url below and use it in the search:
https://m.media-amazon.com/images/I/614tCTX0LeL._AC_SX569_.jpg

3.5.5 MongoDB Atlas Developer Data Platform

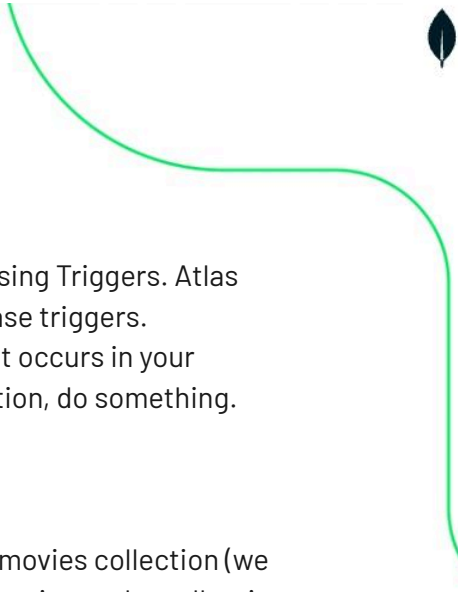
MongoDB Atlas simplifies application development and working with data for a wide variety of use cases, scales globally, and optimizes for price/performance as your data needs evolve over time.

MongoDB Atlas is a developer data platform where you can address the needs of modern applications faster to accelerate your development timelines and reduce data infrastructure complexity.

Apart from Vector search, and the text-full search powered by Lucene it also offers Device Sync and App Services functionality to facilitate application Development.



Atlas Application Services are fully-managed backend services and APIs that help you build apps, integrate services, and connect to your Atlas data faster.



One of the features of app services is to respond to events automatically using Triggers. Atlas includes different types of triggers: authentication, scheduled, and database triggers. This last type allows you to set events and execute functions when an event occurs in your databases. For instance, everytime a new document is inserted in a collection, do something.

3.5.5.1 Using Atlas Database Triggers to create vector embeddings

Imagine we want to include new movies into our `sample_mflix.embedded_movies` collection (we worked with in the 3.5 Using Atlas Vector Search exercise). If we add new movies to the collection just by providing the plot in plain text. Those new movies will not include the `plot_embedding_voyage_3_large` field as therefore will not be part of the vector search.

Remember this collection uses the `plot_embedding_voyage_3_large` field to store the vector embeddings generated using the Open AI's `text-embedding-ada-002` embedding model from the `plot` field.

We used that field when defining the atlas vector search index for this collection to semantically search for movies based on a user description.

We can use Atlas Database Triggers to make sure every time a new movie is inserted in the collection, the plot of the movie is encoded and added to the document that gets inserted.

Let's implement this functionality in our Atlas project.

Make sure your Atlas cluster includes the `sample_mflix.embedded_movies` collection. If not, make sure you Load the sample dataset in your cluster.

In the same project where your Atlas cluster is, create a Database Trigger to set an event on "Insert Document" to the collection and execute a function if that event occurs.

Note: In order to encode the plot using the Open AI's `text-embedding-ada-002` embedding model we will use our MDB Azure AI endpoint. You will need to use the api key provided in this course.

1. Add a Trigger called `AddEmbeddingsTrigger` and configure the trigger settings:
 - Select your Cluster Name.
 - Database Name: `sample_mflix`
 - Collection Name: `embedded_movies`
 - Operation Type: `Insert Document`
 - Full Document: `On`
 - Replace the code with the provided Trigger code.
 - Replace `<API_KEY>` with your OpenAI `API_KEY`

**//Generates a vector embedding of the provided input string.
//model: voyage-4-large***

```
exports = async function (changeEvent) {
  const _id = changeEvent.fullDocument._id;
  const plot = changeEvent.fullDocument.plot;

  // Get the MongoDB service you want to use (see "Linked
  Data Sources" tab)
  // Note: In Atlas Triggers, the service name is defaulted
  to the cluster name.
  const serviceName = "Cluster0";
  const database = "sample_mflix";
  var collName = "embedded_movies";

  // Get a collection from the context
  var collection =
context.services.get(serviceName).db(database).collection(collName);

  var VOYAGE_AI_KEY =
"pa-9FAAbj5Z5od4-tBdVZ5HXhsdEjZxTh3oCpU6kjPDxhP"

  const arg = {
    url: "https://api.voyageai.com/v1/embeddings",
    headers: {
      "Content-Type": ["application/json"],
      "Authorization": [`Bearer ${VOYAGE_AI_KEY}`],
    },
    body: JSON.stringify({
      input: [plot],           // array of strings
      model: "voyage-4-large", // or voyage-4,
voyage-4-lite, etc.
      input_type: "document",  // better for documents
in retrieval use cases
    }),
  };

  const response = await context.http.post(arg);
  // The response body is a BSON binary object. Parse it to
  JSON.
  const responseBody = EJSON.parse(response.body.text());
  const embeddings = responseBody.data[0].embedding;
```

```

    try {
      // Update the document with the new embeddings
      const result = await collection.updateOne(
        { _id: _id },
        { $set: { plot_embedding: embeddings } }
      );
      // Access the number of documents modified
      const modifiedCount = result.modifiedCount;
      //console.log(`Modified ${modifiedCount}
document(s)`);
      return { result: modifiedCount };

    } catch (err) {
      console.log("Error occurred while executing
updateOne:", err.message);
      return { error: err.message };
    }
  };
};

```

2. Test this using your mongosh terminal:

```

mongosh "mongodb+srv://<your_cluster>/" --apiVersion 1
--username <username>

use sample_mflix

db.embedded_movies.insertOne({"title": "The Creator",
"plot": "As a future war between the human race and
artificial intelligence rages on, ex-special forces agent
Joshua is recruited to hunt down and kill the Creator, the
elusive architect of advanced AI.", "imdb": { "rating":
8.9}})

db.embedded_movies.find({"title": "The Creator"})

```

3.6 MongoDB's applications as a Vector Database

So far you've learnt about how Atlas Vector Search works, how it is powered, and tried your hand at a real-world example. The following section covers some of the common applications which when powered by vector databases such as MongoDB, can exponentially enhance the capabilities of the system.



3.6.1 Large Language Models (LLMs)

Large Language Models are designed to understand and generate human-like text. They are trained on massive amounts of text data, which enables them to learn the structure, grammar, and nuances of human language. LLMs can perform a wide range of tasks, such as answering questions, summarizing text, translating languages, and even generating creative content like stories or poems.

3.6.1.1 How do LLMs work?

LLMs are based on a type of neural network architecture called the Transformer. The Transformer architecture is the foundation for many state-of-the-art language models. The key innovation of the Transformer is its ability to process input text in parallel, rather than sequentially, which allows it to scale to much larger datasets.

LLMs are trained using a technique called unsupervised learning, and they learn to generate text by predicting the next word in a sentence, given the words that came before it. During training, the model is fed a large dataset of text and learns to generate text that is similar to the input data. The more data the model is trained on, the better it becomes at understanding and generating human-like text.

3.6.1.2 How can LLMs leverage MongoDB?

Once a Large Language Model has access to a vector database such as MongoDB, it can perform various operations such as similarity comparison, analogy completion, clustering, and search, as demonstrated earlier in the training. This allows the model to generate more accurate responses or retrieve relevant information based on similarities.

In text generation use cases, a Large Language Model (LLM) can make use of MongoDB as shown in the following examples:

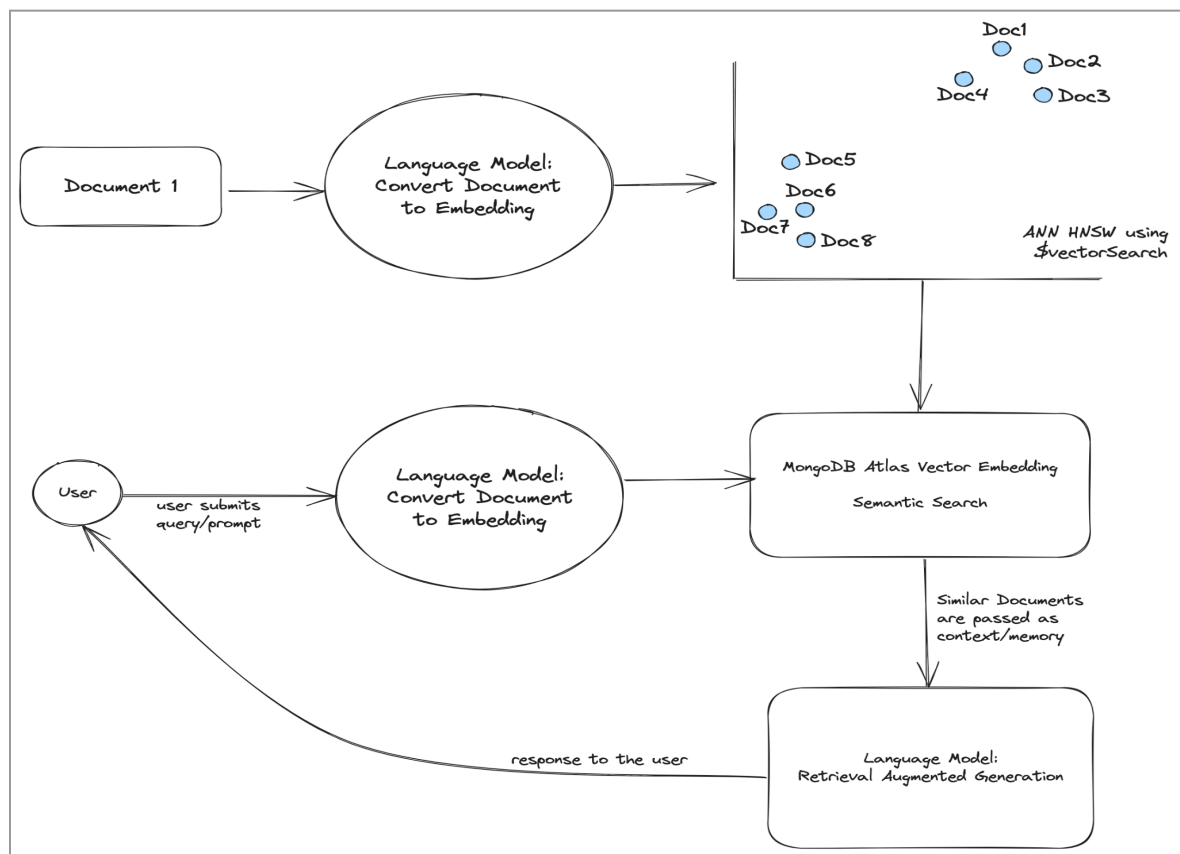
1. Contextual expansion: The LLM can use MongoDB to expand upon a given text prompt and generate more contextually relevant content. By comparing the embeddings or vectors of the prompt with the vectors in the database, the LLM can retrieve similar or related information that can be used to generate more coherent and accurate responses.
2. Sentence completion: In scenarios where the LLM needs to complete an incomplete sentence, MongoDB can help by providing potential options for the missing part. By comparing the vectors of the available text with the vectors in the database, the LLM can identify potential sentence completions that match the context or style of the input.
3. Textual similarity and paraphrasing: By leveraging MongoDB, the LLM can identify similar sentences or phrases from the database that can be used to paraphrase or rewrite the input text.



This is useful for tasks such as text summarization, where the LLM needs to generate a concise version of the input without losing the main meaning.

4. Domain-specific knowledge: MongoDB can contain domain-specific knowledge, allowing the LLM to generate text that is specific to a certain topic or field. By retrieving relevant vectors from the database, the LLM can incorporate domain-specific vocabulary, facts, or patterns into its generation process.

Building and utilizing a vector database requires expertise in natural language processing and data engineering. While this overview provides a high-level understanding, implementation details can vary depending on the specific use case and requirements.



The above diagram shows a system that uses LLMs and vector search to power semantic search.

The system works as follows:

- The user submits a query to the system.
- The system uses an LLM to generate a vector representation of the query.
- The system searches the vector search index for the vectors that are most similar to the query vector.
- The system returns documents associated with the most similar vectors to the user.

3.6.1.3 Applications of LLMs

A non-exhaustive list of applications of Large Language Models can be found below:

- Text summarization: Generating concise summaries of long documents or articles.
- Machine translation: Translating text from one language to another.
- Sentiment analysis: Determining the sentiment or emotion expressed in a piece of text.
- Question answering: Providing accurate answers to questions based on a given context.
- Text generation: Generating creative content like stories, poems, or even code.
- Conversational AI: Powering chatbots and virtual assistants that can understand and respond to human language.



Additional clarification: It is important to make a distinction between the LLMs and the Text Embedding models or other embedding models we might use for encoding our data.

Embedding Model	Large Language Model
Converts text or unstructured data (video, audio, image, etc) into vector embeddings	Understands and generates natural language
Outputs Arrays of numbers (vector embeddings) - Length of the array is defined by the number of dimensions - The dimensionality depends on the embedding model (i.e, text-embedding-ada-002 model generates a vector with 1536 dimensions)	Outputs text, answers, translations, code, etc.
Example: text-embedding-ada-002	Example: GPT-3.5-turbo

3.6.2 Retrieval-augmented generation (RAG) Architecture

Retrieval-augmented generation architecture is a framework that combines the benefits of both retrieval-based systems and generation-based systems in artificial intelligence systems.

In retrieval-based systems, responses are generated by selecting an appropriate response from a predefined set of responses. These models excel in providing accurate and reliable responses, but they may lack creativity and flexibility. On the other hand, generation-based models generate responses from scratch, allowing for more creative and diverse output. However, they may produce inaccurate or irrelevant responses.

The retrieval-augmented generation architecture leverages the strengths of both approaches. It incorporates a retrieval module that retrieves relevant information or responses from a large knowledge base. This retrieved information is then used along with a generation module to produce more accurate and personalized responses.

This architecture is important for several reasons:

1. Better response diversity: By combining retrieval and generation, it allows for a wider range of responses, ensuring diverse and interesting output.

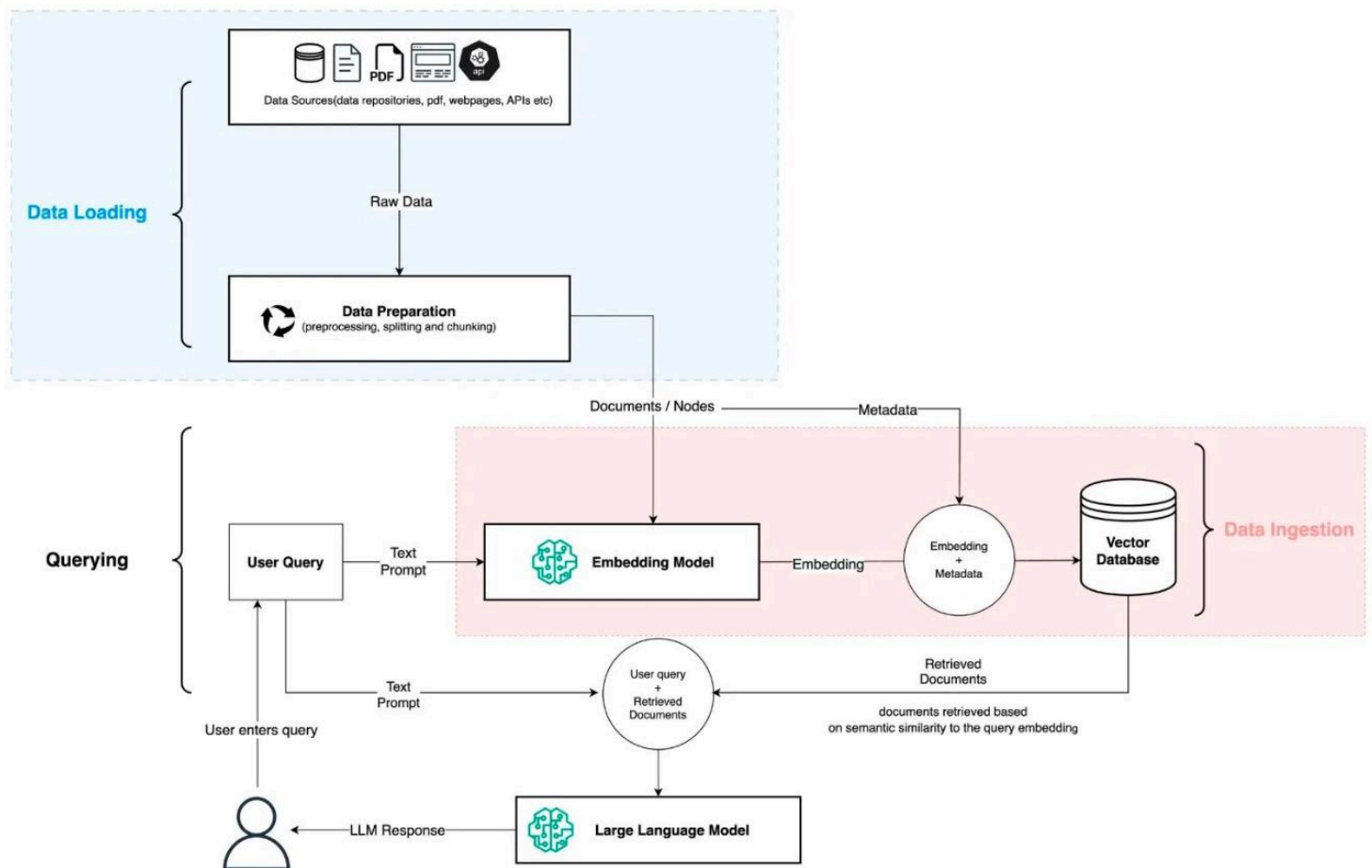


2. More reliable and accurate responses: The retrieval module helps ensure that the generated responses are based on reliable and relevant information, improving the accuracy and quality of the output.

3. Personalization: The retrieval module can be tailored to retrieve information specific to the user or context, allowing for personalized responses.

4. Efficient knowledge utilization: The retrieval module can efficiently search through a large knowledge base, saving computational resources and time compared to generating responses from scratch.

Overall, retrieval-augmented generation architecture enhances the quality, diversity, and personalization of responses, while utilizing knowledge efficiently.

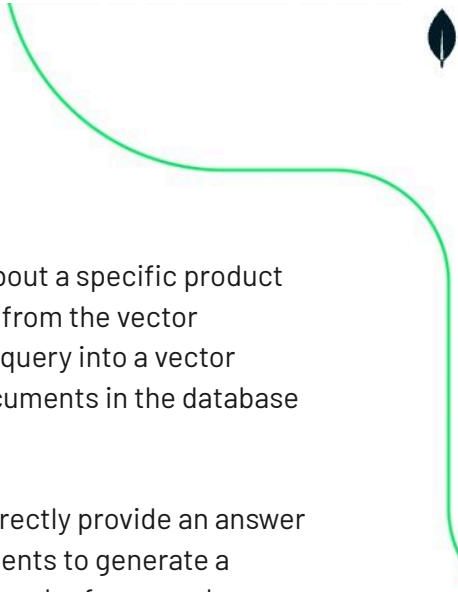


Sample architecture of a typical RAG application

3.6.2.1 Use Case: Customized Chatbot using MongoDB & RAG

In RAG architecture, a vector database, such as MongoDB, plays a crucial role in enhancing the system's performance by facilitating efficient retrieval and generation of relevant information. Let's take an example of a chatbot for customer support.

Suppose you are working on developing a chatbot to assist a company's customer support team. The chatbot needs to provide accurate and helpful responses to incoming customer queries. To achieve this, you can leverage the RAG architecture. In this architecture, MongoDB acts as a repository of pre-computed vector representations of various documents, such as product manuals, FAQs, support articles, and previous customer conversations. Each document is encoded into a vector representation using techniques like word embeddings or sentence embeddings.



Now, let's say a customer interacts with the chatbot and asks a question about a specific product issue. The chatbot's initial step is to retrieve the most relevant documents from the vector database based on the user's query. This is accomplished by encoding the query into a vector representation and comparing it against the vector representations of documents in the database using similarity measures like cosine similarity.

Once the most relevant documents are retrieved, the chatbot can either directly provide an answer from one of the retrieved documents or utilize the content of these documents to generate a response using a generative model like a language model or an encoder-decoder framework.

For example, if a customer asks, "How do I troubleshoot an issue with my laptop battery?", the chatbot would encode this query into a vector representation and search the vector database for documents related to laptop troubleshooting. The documents retrieved could include an FAQ guide or a support article addressing common laptop battery issues.

The chatbot can then either directly extract an appropriate response from the retrieved documents, delivering a pre-existing troubleshooting step, or generate a response by combining or paraphrasing relevant information from multiple retrieved documents.

By employing a vector database such as MongoDB in the retrieval-augmented generation architecture, the chatbot can effectively handle a wide range of customer queries, provide accurate and timely responses, and enhance the overall customer support experience.

3.6.2.2 How does RAG help LLMs?

RAG architecture helps address the following limitations in LLMs:

- RAGs minimize hallucinations by grounding the model's responses in factual information.
- By retrieving information from up-to-date sources, RAG ensures that the model's responses reflect the most current and accurate information available.
- While RAG does not directly give LLMs access to a user's local data, it does allow them to utilize external databases or knowledge bases, which can be updated with user-specific information.
- Also, while RAG does not increase an LLM's token limit, it does make the model's use of tokens more efficient by retrieving only the most relevant documents for generating a response.

3.6.2.3 Exercise: RAG Code Review and Discussion

There are different RAG architectures and examples we can review.

In this case, we will focus on reviewing this RAG example:

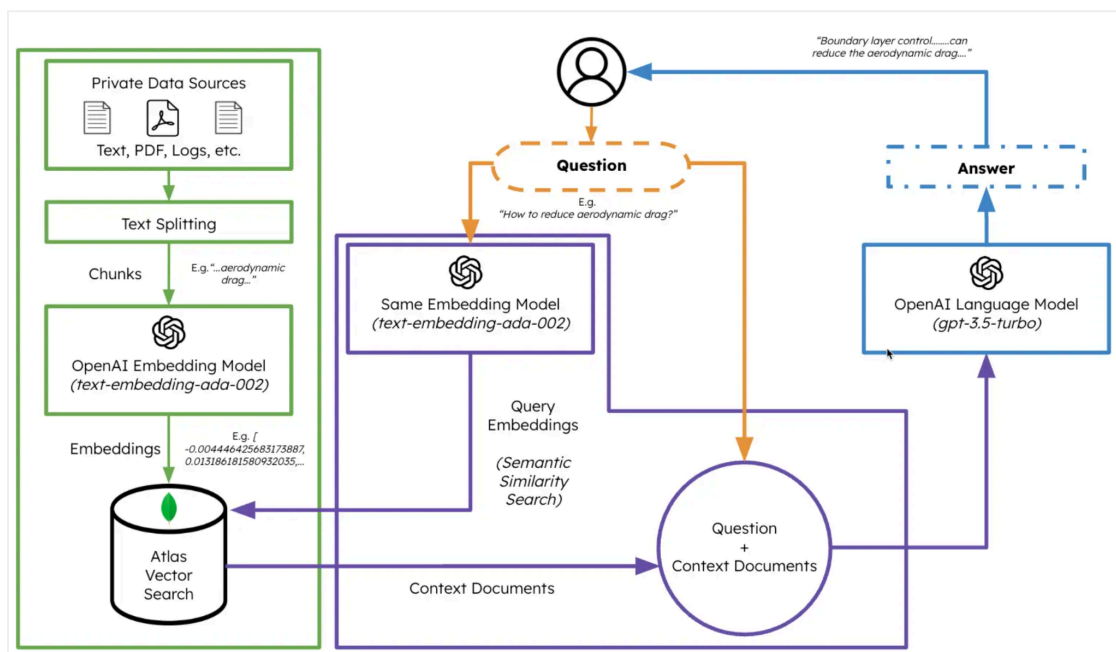
<https://github.com/mongodb-developer/atlas-vector-search-rag>

This code implements a Chatbot using OpenAI and MongoDB. The main components in this example are:

- OpenAI's GPT-3.5 LLMs and text-embedding-ada-002 embedding model. These are used to process natural language questions and create vector embeddings in order to understand their context and semantic meaning, as well as use the natural language processing and generate answers in a natural language format.

Note: An OpenAI API Key token is required for this scenario, as the code interacts with OpenAI. An Open AI paid account will be required for this implementation.

- MongoDB Atlas as the Vector database with Vector indexes with the vector search functionality. This allows us to store all the data that is relevant to provide the context and background information to the Chat Box in order to provide accurate answers to the questions.
- LangChain framework to simplify the creation of LLM applications. It provides a standard interface for chains, lots of integrations with other tools, and end-to-end chains for common applications. This allows AI developers to build LLM applications that leverage external sources of data (for example, private data sources).
- For the User interface we also use Gradio. This is an open-source python library which allows the quick creation of User interfaces for machine learning models.





Setup

1. In a terminal, change to the folder for this exercise.

```
cd /root/da660/atlas-vector-search-rag
```

2. Edit `key_param.py` . **TODO** and insert your MongoDB Connection String and the API key that was given to you by the instructor.
3. Rename the file to `key_param.py`

Load Data into Atlas

Look through the file `load_data.py` and fill out the following table:

Name of the Database?	langchain_demo
Name of Collection?	collection_of_text_blobs
Name of Vector Search Index?	vector_index
Name of field which will contain the vector?	embedding
PDF file to read?	/sample_files/MongoDBDeveloperTrainingProgramDatasheet_UpdatedMay2024.pdf
Size of each chunk?	1000 characters
Overlap between chunks?	20 characters
Embedding model to use to create vectors?	voyage-3-large

Run `load_data.py` to load the data

```
root@mongodb:~/da660/atlas-vector-search-rag# python3 load_data.py
100%|████████████████████| 3/3 [00:04<00:00, 1.57s/it]
```

Create Vector Search Index

Use the following code to create a Vector Search Index in the `langchain_demo` database and the `collection_of_text_blobs` collection through the Atlas UI (name it as `vector_index`):


```
{
  "fields": [
    {
      "numDimensions": 1024,
      "path": "embedding",
      "similarity": "dotProduct",
      "type": "vector"
    }
  ]
}
```

Note

We are creating the index *after* we load the data. This is simply for convenience so that we don't have to create the collection in the UI. **langchain_mongodb.MongoDBAtlasVectorSearch.add_documents()** will update the Vector Search Index if it is there, but will not complain if the index does not exist. The indexing process runs asynchronously (like Atlas Search) so it has no effect on data loading speed.

However, if you change the name of the Vector Search index from "vector_index" you will need to update the code in `extract_information.py`

Pass Vector Search results to GPT

Look through the file `extract_information.py` and fill out the following table:

Embedding Model	voyage-3-large
Chat model	gpt-35-turbo
Number of times the database is queried?	2 - lines 66 and 71 when the chain is invoked

For RAG we "chain" our services together to use the results of one as the inputs for another. The user interface allows us to see what information will be passed to the GPT chat model to create the response.

You can run `extract_information.py` which creates a simple UI at `http://127.0.0.1:7860`

NB: If you are using the virtual Instruqt Lab, you will need to visit the public URL instead in a new browser window:

```
(venv) root@mongodb:~/da660/python/atlas-vector-search-rag# python3 extract_information.py
Running on local URL: http://127.0.0.1:7860
Running on public URL: https://9033e0069d5ff523b3.gradio.live

This share link expires in 72 hours. For free permanent hosting and GPU upgrades, run `grad
```

Note the data that we have loaded: a log entry, a chat transcript, a textbook entry, and a wikipedia article. Try the following queries and review the results:

Question list:

1. Did any error occur on August 16th? If yes, how could the error be corrected?
2. What questions did Alfred ask? What were Bruce's answers? Provide a summary in bullet points.
3. What is yaw?

Note

Question 3 could be answered by something like ChatGPT without needing additional information. The additional information ensures that the GPT model will respond with information that we know to be *accurate*.

The PDF contains the text from MongoDB Instructor-Led Training Datasheet for 2024. Test this out by asking some generic questions about course offerings without mentioning "MongoDB". If you have an OpenAI account, you could compare the responses from ChatGPT and our RAG architecture when you ask the same question.

Some example questions you might use:

- Which course is specifically about aggregation?
- What are the fundamentals courses and who is the intended audience?
- Which training workshop is intended for developers who have completed MDB300?
- Which courses are for Atlas Vector Search?
- Where can I learn about performance optimization?

LLM Hallucination	Which training workshop is intended for developers who have completed MDB300 (MongoDB Production Readiness) The training workshop intended for developers who have completed MDB300 (MongoDB Production Readiness) is MDB400 (MongoDB Advanced Deployment and Operations). This workshop is designed to build on the knowledge and skills acquired in MDB300, focusing on advanced topics related to deploying and managing MongoDB in production environments. MDB400 typically covers areas such as: - Advanced deployment strategies - Performance tuning - Backup and recovery - Security best practices - Scaling and sharding - Monitoring and troubleshooting For the most accurate and detailed information, you should refer to the specific training provider's website or contact their support to ensure that MDB400 is the next recommended step after completing MDB300.
Accurate response with RAG	Question Answering App using Atlas Vector Search + RAG Architecture Enter your Question: Which training workshop is intended for developers who have completed MDB300 (MongoDB Production Readiness) Submit Output with just Atlas Vector Search (returns text field as is): This training workshop is intended for developers who have completed MDB300 (MongoDB Production Readiness) and want to fully understand where a distributed document database necessitates important additional design decisions. This workshop is for those who care about performance and correctness and how to achieve both. MongoDB Training for Developers: Skill Specialty Course Nearly all MongoDB production systems are highly available and distributed; even Output generated by chaining Atlas Vector Search to Langchain's RetrieverQA + OpenAI LLM: The training workshop intended for developers who have completed MDB300 (MongoDB Production Readiness) is DEV400: MongoDB Developer Extension.

4. Vector Search Nodes

Search Nodes provide dedicated infrastructure for Atlas Search and Vector Search workloads, allowing optimization for compute resources and fully scale search needs independent of the database. Search Nodes provide the following advantages:

- Workload isolation
- Better performance at scale (40% - 60% decrease in query time for many complex queries)
- Higher availability

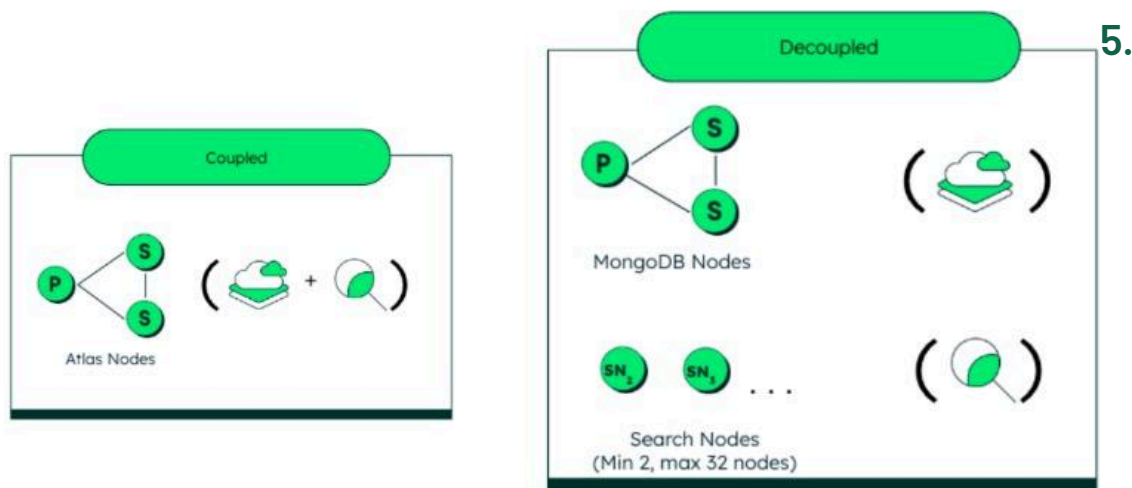
- Improved developer experience

Search Nodes are currently available on AWS single-region clusters (with Google Cloud and Azure coming soon). We can continue using shared infrastructure for Google Cloud and Microsoft Azure for our database collections. You can deploy a minimum of 2 and a maximum of 32 search nodes on your Atlas cluster.

The following instructions guide you on how to configure search nodes in your Atlas cluster (for M10+ clusters):

1. Navigate to your "Database Deployments" section in the MongoDB UI
2. Click the green "+Create" button
3. On the "Create New Cluster" page, change the radio button for AWS for "Multi-cloud, multi-region & workload isolation" to enabled
4. Toggle the radio button for "Search Nodes for workload isolation" to enable.
5. Select the number of nodes in the text box
6. Check the agreement box
7. Click "Create cluster"

Coupled Architecture (left) compared with the decoupled Search Node architecture (right):



Conclusion

By the end of this course, you should have a clear understanding of the artificial intelligence ecosystem comprising the key algorithms, and components. The Movie Recommendation exercise and the Product Image Search exercise help you understand how to use the new Vector Search functionality of Atlas Search, to make use of all the concepts learned in this course, and create a real world example.

What's Next? This course just provided an introduction to GenAI and Vector Search. Now it is time to explore more use cases, new challenges and use Atlas with your own data and vector search.

Here are some additional references and links you can use:

MongoDB embeddings Dataset: For practising or mimic production data volumes with search:
Tech news dataset (1,000,000+) - subset of the Hackernoon dataset - that contains an embedding field for each data point based on the description field (used OpenAI's text-embedding-3-small)

Building A RAG System with Gemma, MongoDB and Open Source Models

Step by step guide using Google's Gemma and open source embedding models with Atlas Vector search. This is part of the Open source AI Cookbook with a collection of notebooks illustrating practical aspects of building AI applications and solving various machine learning tasks using open-source tools and models.

How to Build a RAG System With LlamaIndex, OpenAI, and MongoDB Vector Database

Building an end-to-end RAG system implementing the POLM stack, which leverages embedding models and LLMs provided by OpenAI.

Solutions and Troubleshooting

Embedded VS Code

The Instruqt image uses 'code-server' from <https://github.com/coder/code-server> which is normally started as a service. As we don't have systemd we start it during the provisioning process.

If for any reason it fails, refreshing the tab or even the entire window should bring it back. In an emergency, you can kill the code-server process and relaunch:

```
# killall code-server
# nohup /usr/bin/code-server --bind-addr 0.0.0.0:8081 --auth none
--disable-telemetry --disable-update-check >> /tmp/code-server.log
2>&1 &
```

Similar Image search

flask_server.py

```
from sentence_transformers import SentenceTransformer, util

from flask import Flask, render_template, request, jsonify
import os, sys
import pymongo
import ssl
from bson import json_util
import json
from PIL import Image
import requests
from io import BytesIO

sys.path.insert(1, '../config/')
from config_database import mongo_uri, db, collection

app = Flask(__name__,
            static_url_path='',
            static_folder='../encoder',)

connection = pymongo.MongoClient(mongo_uri)
product_collection = connection[db][collection]
preTrainedModelName = "clip-ViT-L-14"
model = SentenceTransformer(preTrainedModelName)

@app.route('/similarSearch', methods=['GET'])
def searchSimilar():
    vector_img_path = request.args.get('vector', default=None,
    type=str)
    if vector_img_path.startswith("http"):
        img = Image.open(requests.get(vector_img_path,
```

```
stream=True).raw)
    else:
        img = Image.open(vector_img_path)

    vector_img_query = model.encode(img).tolist()

    pipeline = [
        {
            "$vectorSearch": {
                "index": "vectorImg",
                "path": "imageVector",
                "queryVector": vector_img_query,
                "numCandidates": 150,
                "limit": 10
            }
        },
        {
            "$project": {
                "imageVector": {"$slice": ["$imageVector", 5]},
                "imageFile": 1,
                "price": 1,
                "discountPercentage": 1,
                "averageRating" : 1,
                "_id": 0,
                'score': {
                    '$meta': "vectorSearchScore"
                }
            }
        }
    ]
```

```

# Execute the pipeline
docs = list(product_collection.aggregate(pipeline))

# Return the results unders the docs array field
json_result = json_util.dumps({'docs': docs},
json_options=json_util.RELAXED_JSON_OPTIONS)

return jsonify(json_result)

@app.route('/search', methods=['GET'])
def search():

    vector_text = request.args.get('vector', default=None,
type=str)
    vector_query = model.encode(vector_text).tolist()
    pipeline = [
        {
            "$vectorSearch": {
                "index": "vectorImg",
                "path": "imageVector",
                "queryVector": vector_query,
                "numCandidates": 150,
                "limit": 10
            }
        },
        {
            "$project": {
                "imageVector": {"$slice": ["$imageVector", 5]},

```



```

        "imageFile": 1,
        "price": 1,
        "discountPercentage": 1,
        "averageRating" : 1,
        "_id": 0,
        'score': {
            '$meta': "vectorSearchScore"
        }
    }
}

]

# Execute the pipeline
docs = list(product_collection.aggregate(pipeline))

# Return the results unders the docs array field
json_result = json_util.dumps({'docs': docs},
json_options=json_util.RELAXED_JSON_OPTIONS)

return jsonify(json_result)

@app.route('/searchAdvanced', methods=['GET'])
def searchAdvanced():

    vector_text = request.args.get('vector', default=None,
type=str)
    minRating = request.args.get('minRating', default=3,
type=float)
    minDiscount = request.args.get('minDiscount', default=5,
type=float)
    maximumPrice = request.args.get('maxPrice', default=75,
type=float)

```

```
sortBy = request.args.get('sortBy', default=None, type=str)

print(f"Parameters: {vector_text}, {minRating}, {minDiscount},
{maximumPrice}, {sortBy}")

sort = {"score": -1}

if (sortBy=="price"):
    sort = {"price":1}
elif (sortBy=="averageRating"):
    sort = {"averageRating":-1}

vector_query = model.encode(vector_text).tolist()
pipeline = [
    {
        "$vectorSearch": {
            "index": "vectorImg",
            "path": "imageVector",
            "queryVector": vector_query,
            "numCandidates": 150,
            "limit": 10,
            "filter":{
                "$and":[
                    {
                        "price": {
                            "$lt": maximumPrice
                        }
                    },
                    {
                        "averageRating": {
                            "$gte": minRating
                        }
                    }
                ]
            }
        }
    }
]
```

```

        },
        {
            "discountPercentage": {
                "$gte": minDiscount
            }
        }
    ]
}
},
{
    "$project": {
        "imageVector": {"$slice": ["$imageVector", 5]},
        "imageFile": 1,
        "price": 1,
        "discountPercentage": 1,
        "averageRating" : 1,
        "_id": 0,
        'score': {
            '$meta': "vectorSearchScore"
        }
    }
},
{
    "$sort" : sort
}

]

# Execute the pipeline
docs = list(product_collection.aggregate(pipeline))

```

```

    # Return the results unders the docs array field
    json_result = json_util.dumps({'docs': docs},
    json_options=json_util.RELAXED_JSON_OPTIONS)

    return jsonify(json_result)

# page
@app.route('/')
def index():
    return render_template("index.html")

@app.route('/advanced')
def advanced():
    return render_template("advanced.html")

@app.route('/similar')
def similar():
    return render_template("similar.html")

if __name__ == '__main__':
    # app.run(host="localhost", port=5010, debug=True)
    app.run(host="0.0.0.0", port=5010, debug=True)

```

similar.html

```

<link
href="//netdna.bootstrapcdn.com/bootstrap/3.1.0/css/bootstrap.min.
css" rel="stylesheet">

<style>

```



```
@import
"http://fonts.googleapis.com/css?family=Roboto:300,400,500,700";

.container {
  margin-top: 20px;
}

.mb20 {
  margin-bottom: 20px;
}

hgroup {
  padding-left: 15px;
  border-bottom: 1px solid #ccc;
}

hgroup h1 {
  font: 500 normal 1.625em "Roboto", Arial, Verdana, sans-serif;
  color: #2a3644;
  margin-top: 0;
  line-height: 1.15;
}

hgroup h2.lead {
  font: normal normal 1.125em "Roboto", Arial, Verdana,
sans-serif;
  color: #2a3644;
  margin: 0;
  padding-bottom: 10px;
}
```

```
.search-result .thumbnail {
  /* border-radius: 0 !important; */
  width: 15em;
}

.search-result:first-child {
  margin-top: 0 !important;
}

.search-result {
  margin-top: 20px;
}

.search-result .col-md-2 {
  border-right: 1px dotted #ccc;
  min-height: 140px;
}

.search-result ul {
  padding-left: 0 !important;
  list-style: none;
}

.search-result ul li {
  font: 400 normal .85em "Roboto", Arial, Verdana, sans-serif;
  line-height: 30px;
}

.search-result ul li i {
  padding-right: 5px;
}
```

```
.search-result .col-md-7 {
  position: relative;
}

.search-result h3 {
  font: 500 normal 1.375em "Roboto", Arial, Verdana, sans-serif;
  margin-top: 0 !important;
  margin-bottom: 10px !important;
}

.search-result h3>a,
.search-result i {
  color: #248dc1 !important;
}

.search-result p {
  font: normal normal 1.125em "Roboto", Arial, Verdana,
sans-serif;
}

.search-result span.plus {
  position: absolute;
  right: 0;
  top: 126px;
}

.search-result span.plus a {
  background-color: #248dc1;
  padding: 5px 5px 3px 5px;
}
```

```
.search-result span.plus a:hover {
  background-color: #414141;
}

.search-result span.plus a i {
  color: #fff !important;
}

/* .search-result span.border {
  display: block;
  width: 97%;
  margin: 0 15px;
  border-bottom: 1px dotted #ccc;
} */

.search-result .meta-search li {
  font-weight: 600;
}

.search-result .meta-search li > span{
  font-weight: 100;
}

.highlight {
  background-color: #a94442;
  color: white;
  padding-left: 5px;
  padding-right: 5px;
}

</style>
```



```

<html>
<div class="container">
  <div class="row">
    <div class="col-12">
      <h1>Demo: MongoDB Atlas Vector Search on Product Images</h1>
    </div>
  </div>
</div>

<div class="container">
  <div class="row">
    <div class="col-xs-1">
      <div style="padding-left: 1px;">
        <p>Product:</p>
      </div>
    </div>
    <div class="col-xs-2">
      <div style="padding-left: 5px;">
        <div id="custom-search-input" class="input-group
custom-search-form">
          <input id="search_box" type="text" class="typeahead"
placeholder="search by keyword">
        </div><!-- /input-group -->
      </div>
    </div>
    <div class="col-xs-1">
      <button type="button" class="btn btn-success"
id="search_button">Search</button>
    </div>
  </div>
</div>

```

```

    </div>
</div>

<div class="container">

    <hgroup class="mb20">
        <h1>Search Results</h1>
        <h2 class="lead"><strong class="text-danger"
id="result-count">0</strong> results were found for the search for
<strong class="text-danger" id="search-query">____</strong></h2>
    </hgroup>

    <section class="col-md-12"
id="search-results-placholder"></section>
</div>

</html>

<script
src="https://twitter.github.io/typeahead.js/js/jquery-1.10.2.min.js"></script>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/corejs-typeahead/1.2.1
/typeahead.bundle.min.js"></script>

<script>

    $("#search_box").keydown(function(e){
    if(e.which === 13){
        $("#search_button").click();
    }

```

```
});

$('#search_button').click(function() {
    var letters = $('#search_box').val();

    $.ajax({
        url: `/similarSearch?vector=${letters}&path=title`
    }).done(function(results){
        var jsonResults = JSON.parse(results).docs
        console.log(jsonResults)
        render(letters, jsonResults)
    })
    .fail(function(err) {
        console.log(err)
    });
});

});
```

```
function render(letters, results) {
    var placholder = $('#search-results-placholder');
    placholder.empty();

    // update count
    $('#result-count').html(results.length)
    // update search query
    $('#search-query').html(letters)

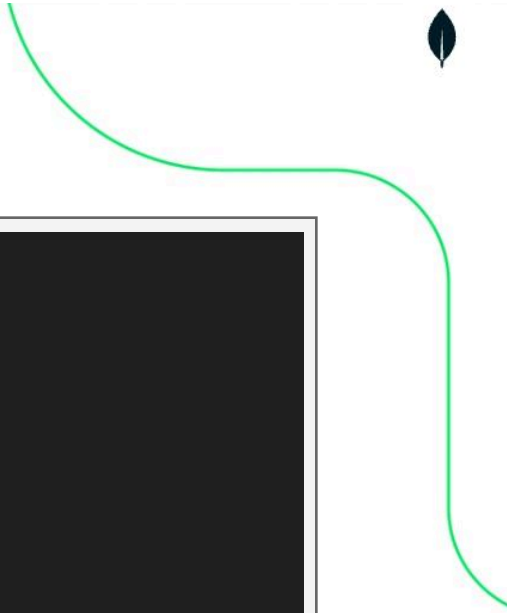
    let html = '';
    $.each(results, function(index, item) {
```

```

    let doc = item;
    if (doc.poster == undefined) {
        doc.poster = "{{
url_for('static',filename='no-picture.jpeg') }}"
    }

    html += `
<article class="search-result row">
    <div class="col-md-3">
        <div class="thumbnail"></div>
    </div>
    <div class="col-md-2">
        <ul class="meta-search">
            <li>Image File: <span>${doc.imageFile}</span></li>
        </ul>
    </div>
    <div class="col-md-7 excerpt">
        <li><strong>The data in the database:</strong></li>
        <li><strong>Price:
</strong><span>${doc.price}</span></li>
        <li><strong>Average Rating:
</strong><span>${doc.averageRating}</span></li>
        <li><strong>Discount Percentage:
</strong><span>${doc.discountPercentage}%</span></li>
        <li><strong>Search score: </strong>${item.score}</li>
        <li><strong>Embeddings (only 5 dimensions
displayed):</strong><span>[${doc.imageVector}...]</span></li>
    </div>
    <span class="clearfix borda"></span>
</article>
`

```



```
});  
// place in html  
placeholder.append(html);  
}  
  
$(document).ready(function() {  
    //search();  
});  
</script>
```